MITIGATION OF CROP DISEASE DETECTION

Thesis/Dissertation submitted in the partial fulfillment of the requirements for the awardof the degree of

BACHELOR OF TECHNOLOGY
in
CSE (DATA SCIENCE)

by

N. KARTHIK REDDY	**21K95A6708**
G. NEETHA REDDY	**20K91A6706**
K. MADHUSUDHAN KUMAR	**20K91A6717**
D. SUMALINI GOUD	**20K91A6749**

Under the guidance of
Dr. RAJESH BANALA
Associate prof. of CSE (Data Science)

DEPARTMENTMENT OF COMPUTER SCIENCE ENGINEERING (DATA SCIENCE)
TKR COLLEGE OF ENGINEERING &TECHOLOGY (AUTONOMOUS)
(Accredited by NAAC with 'A^{+}' Grade)
Medbowli, Meerpet, Saroornagar, Hyderabad-500097

CERTIFICATE

This is to certify that the project report entitled **"MITIGATION OF CROP DISEASE DETECTION"**, being submitted by **Mr. N. Karthik Reddy**, bearing **Roll.No:.21K95A6708, Ms. G. Neetha Reddy**, bearing **Roll.No:.20K91A6706, Mr. K. Madhusudhan Kumar**, bearing **Roll.No:.20K91A6717 and Ms. D. Sumalini Goud**, bearing **Roll.No:.20K91A6749** in partial fulfillment of requirements for the award of degree of **Bachelor of Technology in *CSE(Data Science)***, to the TKR College of Engineering & Technology is a record of bonafide work carried out by them under my guidance and supervision.

Co-Ordinator
Mr. M. Arokia Muthu
Assistant Professor

Head of Dept. of CSE(DS)
Dr. V. Krishna
Professor

Internal Guide
Dr. Rajesh Banala
Associate Professor

EXTERNAL EXAMINAR

DECLARATION BY THE CANDIDATES

We, **Mr. N. Karthik Reddy** bearing Hall Ticket Number: **21K95A6708**, **Ms. G. Neetha Reddy** bearing Hall Ticket Number: **20K91A6706, Mr. K. Madhusudhan Kumar** bearing Hall Ticket Number: **20K91A6717** and **Ms. D. Sumalini Goud** bearing Hall Ticket Number: **20K91A6749** hereby declare that the major project report titled **MITIGATION OF CROP DISEASE DETECTION** under the guidance of **Dr. RAJESH BANALA** , ASSOCIATE PROFESSOR in Department of Computer Science &Engineering is submitted in partial fulfillment of the requirements for the award of the degree of ***Bachelor of Technology in CSE(Data Science)***.

N. Karthik Reddy 21K95A6708

G. Neetha Reddy 20K91A6706

K. Madhusudhan 20K91A6717

D. Sumalini Goud 20K91A6749

ACKNOWLEDGEMENT

The satisfaction and euphoria that accompanies the successful completion of any task would be incomplete without the mention of the people who made it possible and whose encouragement and guidance have crowned our efforts with success.

We express our sincere gratitude to **Management of TKRCET** for granting permission and giving inspiration for the completion of the project work.

Our faithful thanks to our **Principal Dr. D. V. Ravi Shankar, M.Tech., Ph.D.,** TKR College of Engineering & Technology for his Motivation in studies and completion of the project work.

With our heart full pleasure we thank our **Head of the Department Dr. V. Krishna, M.Tech., Ph.D.,** Professor, Department of CSE (Data Science), TKR College of Engineering & Technology for his suggestions regarding the project.

Thanks to our Project Coordinator **Mr. M. Arokia Muthu, M.E., Ph.D.,** Assistant Professor, Department of CSE (Data Science), TKR College of Engineering & Technology for his constant encouragement.

We are indebted to the **Internal Guide, Dr. Rajesh Banala, M.Tech., Ph.D.,** Associate Professor, Department of CSE (Data Science), TKR College of Engineering & Technology for his support in completion of the project successfully.

Finally, we express our thanks to one and all that have helped us in successfully completing this project. Furthermore, we would like to thank our family and friends for their moral support and encouragement.

By,

N. Karthik Reddy 21K95A6708

G. Neetha Reddy 20K91A6706

K. Madhusudhan 20K91A6717

D. Sumalini Goud 20K91A6749

CONTENTS

ABSTRACT

The importance of agriculture in terms of development is invaluable as it provides food to over 58% percent of the world population. However, mainstream farming techniques face challenges, especially for disease detection that has necessitated the use of modern alternatives. This paper investigates the combination of image processing and deep learning methods that have been employed in automated plant disease diagnosis. Our approach involves image acquisition, pre-processing, segmentation, feature extraction and classification with special focus on using Convolutional Neural Network (CNN) models. Additionally, our proposed architecture incorporates parallel attention mechanism modules and residual blocks which improve upon accuracy and efficiency at real time level. By thorough experimentation done by our CNN model, crop disease image classification attains a remarkable 99.17% accuracy. These findings demonstrate the potential of CNNs for real-world applications and open up new avenues to explore concerning pests' identification and plant disorders recognition.

KEYWORDS: Agriculture, Disease detection, CNN, Accuracy

LIST OF FIGURES

LIST OF TABLES

LIST OF SCREENS

SYMBOLS &ABBREVIATIONS

Acronym	Expansion
CNN	**Convolutional Neural Network**
IPM	**Integrated Pest Management**
IoT	**Internet of Things**
DL	**Deep Learning**
API	**Application Programming Interface**
RGB	**Red Green Blue**
GLCM	**Grey Level Co- Occurrence Matrix**
VNIR	**Visible and Near Infrared**
SWIR	**Short-Wave Infrared**
BPNN	**Back Propagation Neural Network**
KNN	**K-Nearest Neighbors**
SVM	**Support Vector Machine**
RNN	**Recurrent Neural Network**
RBAC	**Role Based Access Control**
ML	**Machine Learning**
HTTP	**Hypertext Transfer Protocol**
SPP	**Spatial Pyramid Pooling**
ReLU	**Rectified Linear Unit**
GPU	**Graphics Processing Unit**
GUI	**Graphical User Interface**

1. INTRODUCTION

1.1 Motivation

Agriculture, often referred to as the foundation of economies plays a role, in supporting livelihoods and ensuring food security. At the core of success lies crop production, which depends on factors like managing pests applying fertilizers effectively and using water efficiently. This study aims to explore the interplay among these production elements with a focus on how plant diseases, pest control methods and recommendations for selecting crops and applying fertilizers can contribute to prosperity.

Detecting and managing plant diseases and pests are responsibilities in advancing precision agriculture. These issues are widespread. Pose risks to crop yields and quality. They can result in losses and threaten food availability underscoring the importance of their effective management for sustainable agriculture. By utilizing technologies and innovative approaches such as recognition systems aided by user friendly portable devices the agricultural industry can improve disease and pest monitoring efforts.

Automated disease detection systems offer benefits over methods particularly in terms of efficiency, accuracy and cost effectiveness. These systems leverage advancements, in image processing, machine learning and sensing technologies to rapidly identify and diagnose plant diseases. Through the analysis of images of crop leaves or using sensors to track changes, in plants these technologies are capable of identifying signs of disease emergence or pest invasion. Detecting issues at a stage is crucial, for disease and pest control because it facilitates timely and precise actions to address them.

By using data driven insights and predictive analytics farmers can make informed choices on how to handle pest issues like using biopesticides adjusting farming methods or planting crops that're resistant, to pests. Furthermore embracing pest management (IPM) strategies that focus on maintaining balance and reducing reliance on chemical pesticides can promote sustainable agricultural practices.

The adoption of precision agriculture methods made possible by advancements shows potential for transforming farming practices and supporting sustainable growth. Through the use of data analytics, devices IoT(Internet of Things) and remote sensing technologies farmers can efficiently allocate resources improve crop management techniques and reduce risks related to pest infestations and crop diseases. Additionally educating farmers about farming practices and digital tools is crucial, for maximizing the advantages of precision agriculture initiatives.

1.2 Problem definition

Crop failure can be put down to a host of causes like plant diseases, inadequate pest control applications and inaccurate crop estimates. Ideally, crop forecasting ought to be based on crop observation and quality. In order to stop the same disease next time fertilizers need to used minimally as per detected disease. It plays an important role in the identification and management of plant diseases which is essential for effective plant disease control practices.

1.3 Limitations of existing system

The other problem is that a lot of such currently existing systems are not accurate. But this was done on various platforms for prediction in the existing systems. They obtained an accuracy of 86.1% after training the model. The PestDetNet framework used 11 learnable layers, which include eight convolutional and three fully connected layers. Consequently, they had to employ a number of techniques for preprocessing as well as transfer learning approaches in order to get some accuracy but it’s not within expected limits.

1.4 Proposed system

The primary aim of this project is to propose a DL system that helps in precise plant pathogen and pest control. Our project proposes the right and necessary agriculture roots and provides an online compartment. It is an integrated platform where all plant diseases can be diagnosed on one interface only. It also incorporates FastAPI which lets us predict within seconds This proposed model has high precision and is effective.

2. LITERATURE SURVEY

2.1 PAPER-1

Title: An Efficient Approach for Crops Pests Recognition and Classification Based on Novel DeepPestNet Deep Learning Model

Authors: Naeem Ullah, Javed Ali Khan, Lubna Abdulaziz Alharbi, Asaf Raz

Description:

Naeem Ullah et Al. (2022) Crop pests' correct identification is important to save the economy and environment – the meanings of this sentence are intact. Most existing approaches suffer from poor accuracy because their algorithms are highly complicated, while the data available for them to use is limited. The paper presents DeepPestNet, an end-to-end framework for pest recognition and classification that recorded 100% accuracy on Deng's crops dataset and 96.92% on Kaggle Pest Dataset. Due to possible improvements in image recognition through deep learning models, efforts have been made to develop more accurate systems for pest identification. The suggested model could be a source of immediate relief for professionals as well as farmers, with prospects of reduced economic losses plus increased crop outputs.

Merits:

- Innovation: This paper presents a novel end-to-end DeepPestNet framework designed for pest detection and classification.
- Data Augmentation: It implements image rotations to augment data and increase robustness of models.
- Difference from Traditional Models: The model is compared with other pre-trained deep learning models, showing its efficiency.

Demerits:

- Limited DataSet Availability: A challenge with limited data available for deep learning models training in agricultural applications which is common.
- Complexity: Deep learning models are complex and need computational resources for training and deployment.

2.2 PAPER-2

Title: Plant Disease Detection and Classification by Deep Learning

Authors: Lili Li, Shujuan Zhang, Bin Wang

Description:

Plant leaf diseases detection and classification with deep learning are the subjects explored in this review paper. It is common knowledge that early detection of plant diseases is crucial to agriculture as a whole and conventional methods that rely on manual identification have their own share of problems. The paper then presents an account of deep learning, particularly discussing convolutional neural networks (CNN), with a focus on their efficacy when used for plant disease recognition. Moreover, it highlights the problem of small datasets in training deep learning models for plant disease recognition and suggests that transfer learning can be used as a solution. In addition to this, traditional image processing techniques are compared to those using deep learning; thus highlighting the advantages of the latter.

Merits:

- Comprehensive coverage: The paper's intent is to discuss in detail the recent improvements on plant disease detection by using deep learning methods, including introduction to some basic concepts of deep learning, evaluation metrics used for assessing the models performance, datasets available and methods of augmenting them.
- Incorporation of recent developments: It overcomes the limitations inherent in previous review papers through highlighting issues like visualizations techniques, adjustability of deep learning architectures among other things.

Demerits:

- No Original Research: The paper provides a broad survey of the current literature but does not present any new research findings or experiments performed by the authors.
- Few Challenges Discussed: The paper mentions challenges like lack of data and imbalanced datasets, yet it could have explored more alternative approaches and future studies.

2.3 PAPER-3

Title: Research on Recognition Model of Crop Diseases and Insect Pests Based on Deep Learning in Harsh Environments

Authors: Yong Ai, Chong Sun, Jun Tie, Xiantao Cai

Description:

The paper develops a recognition model that uses convolutional neural networks (CNNs) to automatically distinguish crop diseases and insect pests. It applies Inception-ResNet-v2 model in training, where the cross layer direct edge and multi-layer convolution are used in residual network unit. For research purposes, this dataset is made up of 27 disease images from 10 different crops obtained from the Al Challenger Competition in 2018. The overall recognition accuracy of this model was found to be 86.1% which shows that it is effective. Additionally, these authors designed a WeChat applet for crop disease and insect pest recognition which were tested and proved to give accurate identification as well as corresponding guidance.

Merits:

- Crop disease and insect pests' automatic identification with deep learning, especially CNNs that can greatly reduce economic losses due to pests.
- There is high overall recognition accuracy of 86.1%, thus validating the effectiveness of the proposed model. Enhances farmer accessibility by developing a we chat applet as practical application for real time recognition and guiding.

Demerits:

- No Original Research: The paper provides a broad survey of the current literature but does not present any new research findings or experiments performed by the authors.
- Few Challenges Discussed: The paper mentions challenges like lack of data and imbalanced datasets, yet it could have explored more alternative approaches and future studies.
- No Critical Appraisal: The study mainly concentrates on summarizing previous work rather than critically analyzing the drawbacks or biases in those studies.

2.4 PAPER-4

Title: Pest Detection and Classification in Peanut Crops Using CNN, MFO, and EVITA Algorithms

Authors: P. Venkata Sai ChandraKanth, M. Iyappa Raja

Description:

The reason behind this work is the vast developments in ViT approaches that have shown effective results in image classification tasks. This paper focuses on implementing ViT models by learning dual branch segment representations as a means to enhance accuracy of image classification. In doing so, authors suggest a double-layer transformer encoder that incorporates pest image segments of different sizes thereby improving overall image features. Three pest datasets affecting peanut crops: Aphids, Wireworm, and Gram Caterpillar are used for the study which were collected from publicly available repositories. The MFO technique is used to preprocess the datasets while the linear projection method is employed for image flattening to improve their quality. Then normalization techniques are applied to generate mathematical representation of these images.

Merits:

- Multiple Algorithm Integration: The research mingles the use of CNN, MFO and ViT algorithms to enhance pest detection and classification process, by capitalizing on what each approach is best at.
- An Innovative Concept: The new passage adds more to the existing body of knowledge about crop pest control through developing a model called Enhanced Vision Transformer Architecture (EVITA) specifically for identifying pests in groundnut plants.

Demerits:

- No comparison was made juxtaposing it however claims that this is superior to other contemporary models. A more thorough comparative analysis will help confirm the proposed methodology taken.
- Scarcity of dataset characteristics' discussion: Although the paper mentions its datasets, they are not discussed in detail, which may lead to non-reproducibility and reduced generalizability of outcomes.

2.5 PAPER-5

Title: Automated Plant Leaf Disease Detection using Image Processing Techniques

Authors: Rahul Kundu, Usha Chauhan, S.P.S Chauhan

Description:

The proposed technique is an extension of the past studies in plant disease detection and image processing. Some researches have made attempts to employ computer vision techniques with leaves images for crop diseases identification. These methods include machine learning algorithms, deep learning models, and image segmentation tools that help to study leaf features and identify deviations that might relate to diseases. Furthermore, there has been a great deal of work conducted on building a diverse set of plant species and disease types so as to create reliable databases for training and testing detection models. Furthermore, real-time crop health surveillance through advancements in sensor technologies and data acquisition systems allows for timely intervention measures before an outbreak can occur.

Merits:

- In advance of time: The proposed system enables early detection of plant diseases, which helps farmers to act in good time and avoid crop losses.
- No violation of the body inside: Utilizing image processing techniques, the detection process is non-invasive and does not require physical contact with plants so as not to disrupt their growth.
- It's not expensive at all: Automated disease detection offers a cost-effective solution compared to traditional manual inspection methods, reducing labor and resource expenses for farmers.

Demerits:

- Quality Dependency on Picture: Things like lighting, camera quality, and image resolution might affect the accuracy of disease diagnosis, making it difficult to implement in real life.
- Broad model of Detection: This is how training information and model optimization are necessary to ensure that detection models generalize across different plant species and types of diseases.

2.6 PAPER-6

Title: Plant Leaf Disease Detection and Classification Based on CNN with LVQ Algorithm

Authors: Melike Sardogan, Adem Tuncer, Yunus Ozen

Description:

The paper presents a way of detecting and classifying tomato leaf diseases by using Convolutional Neural Networks (CNN) and the Learning Vector Quantization (LVQ) algorithm. The research studies bacterial spot, late blight, septoria leaf spot, and yellow curved leaf diseases, which severely affect the tomato production. CNN does automatic feature extraction and classification through color from RGB components. The LVQ algorithm is applied to train the network using an output feature vector computed by the convolutional part of CNN. The results of experiments show that the method proposed can accurately recognize four different types of tomato leaf diseases.

Merits:

- Automated Disease Detection: Tomato leaf diseases are detected and classified by this method, which is important in time intervention and crop output.
- Utilization of CNN: CNNs can be used to automatically extract relevant features from pictures of tomato leaves, doing away with the need for manual feature engineering.
- Integration of LVQ Algorithm: As a result, the classification process has been improved through efficient use of the output feature vectors from the CNN by inclusion of the LVQ algorithm.

Demerits:

- The dataset that is much smaller in size can limit the generalization ability of the model for tomato leaf diseases due to its inability to capture all types of variation and diversity with only 500 images.
- In this regard, the methodology's reliability may depend upon how good the photos were taken, such as their resolution, illumination circumstances and any artifactual problems they might exhibit.

3. REQUIREMENTS ANALYSIS

3.1 Functional Requirements

Image Upload and Disease Categorization: Users should be able to upload photographs of plant diseases. When a photograph is uploaded, the system should process it so as to classify the disease correctly. The user is supposed to see the classification results which would come along with information related to that particular disease.

Home Module: There ought to feature advertisement engine in the homepage. This page should contain links leading to three different plant disease classification modules that are Tomato, Bell Pepper and Potato diseases. Every module must have an option for uploading pictures specific for each plant type for disease classification purposes only.

About Module: The About page is expected to have information about technologies applied in classifying diseases. It should also describe animal husbandry procedures as well.

Services Module: This section should outline what farmers can expect from this company's services offer them, It must include statistics like number of crops serviced, villages covered, samples collected plus their locations.

Organic Module: Everything you need to know about traditional organic farming techniques.

Blog Module: The Blog page will contain informative articles on subjects like diseases of plants, farming methods etc.

Details Module: This module must give much information about diverse plant diseases like symptoms, causes, how to prevent them and their cure.

Gallery Module: On the Gallery page, show images about plant diseases, agriculture practices among other relevant visuals.

Contact Module: The company must include a contact section that contains address, phone number and email address.

User Authentication and Management: Users need to create accounts and securely log in. Administrators should have access to control users' accounts as well as content.

Responsive Design: Ensure it can be accessed through different devices such as tablets or mobile phones of varying screen sizes.

Search Functionality: You want users to search for particular diseases, articles on specific topics within your site.

Feedback Mechanism: There is need for feedback system where users are able to give ideas and report problems.

3.2 Non Functional Requirements

Non-Functional Requirement is a quality attribute of a software system. They evaluate the software system's responsiveness, usability, security, portability, and other non-functional characteristics that are critical to its success. Non-functional requirements must be specified with the same attention as :-

Performance: The website should have a response time of less than 2 seconds for basic page loads. Image processing for disease classification should be completed within 5 seconds. The system should handle a peak load of 1000 concurrent users without significant performance degradation.

Scalability: The system architecture should support horizontal scaling to accommodate increased traffic and data volume. It should be able to scale seamlessly across multiple servers or cloud instances.

Reliability: The website should have a minimum uptime of 99.99% per month, excluding scheduled maintenance. Automated monitoring and alerting systems should be in place to detect and respond to any downtime or performance issues promptly.

Security: All user data and image uploads should be encrypted both in transit and at rest. Access to sensitive information should be restricted based on role-based access control (RBAC). Regular security audits and vulnerability assessments should be conducted to identify and address potential security risks.

Data Privacy: The system should comply with data protection regulations such as GDPR, ensuring the confidentiality and integrity of user data. User consent should be obtained before collecting any personal information, and mechanisms should be in place for users to manage their privacy preferences.

Accessibility: The website should adhere to WCAG 2.1 AA standards, ensuring accessibility for users with disabilities. Accessibility features such as alternative text for images and keyboard navigation should be implemented.

Compatibility: The website should be compatible with major web browsers including Chrome, Firefox, Safari, and Edge, across multiple devices and platforms. Responsive design principles should be followed to ensure consistent user experience across different screen sizes and resolutions.

Maintainability: The codebase should be well-organized, following best practices and coding standards. Comprehensive documentation should be provided for developers, including setup instructions, architecture overview, and code comments. Automated testing suites should be in place to facilitate continuous integration and deployment (CI/CD) pipelines.

Localization: The website should support localization for different languages and regions, allowing users to view content in their preferred language and format. Localization should extend beyond content translation to include cultural and regional nuances in design and user experience.

4. DESIGN

4.1 System Architecture

The proposed System architecture comprises of data acquisition from a huge dataset, processing at different convolutional layers and then the classification of plant diseases which declares if the plant image is of a healthy class or diseased class

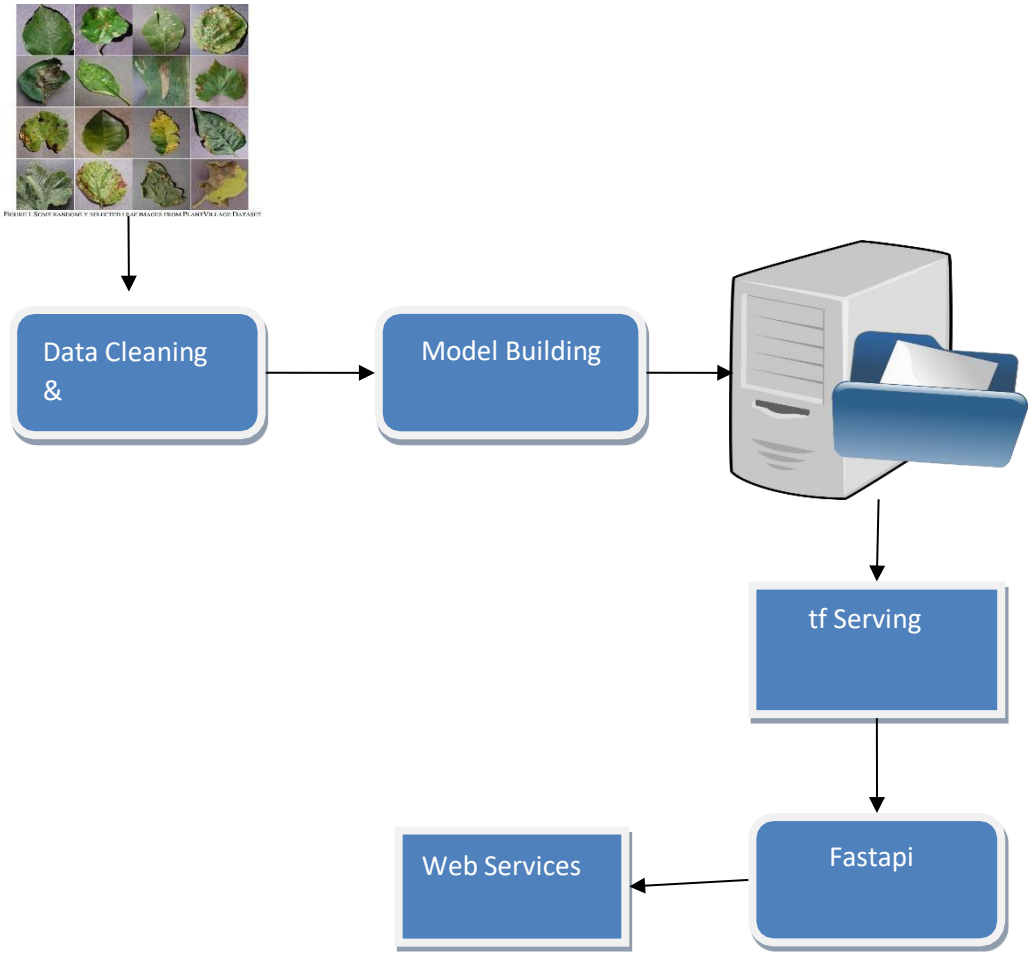


Fig.4.1. Architecture Diagram

Stage 1: A dataset of plant diseases

In this stage, you collect a complete dataset of images that show different plant diseases. For the model to be robust and generalize well, the set should include numerous disease types and plant species. You can source such datasets from multiple areas such as academic research repositories or government databases or even personally by collecting images. Every photo in the dataset should indicate which kind of disease connected with its respective plant species.

Stage 2: Data Cleaning & Preprocessing

Data cleaning involves various tasks that help to ensure your CNN-ready dataset is fit for training your CNN model properly. Some of the actions involved at this step are:

- Removing all worthless and/or corrupted pictures from the data set.
- Resizing images to a consistent size which is suitable for CNN input.
- Normalizing pixel values so that they fall within a common range (usually between 0 and 1).
- The dataset may be enriched through different transformations including rotation, flipping, brightness adjustment, etc., thereby enhancing its diversity and minimizing overfitting risk.

Step 3: Building Models (CNN)

This phase involves the design and training of your CNN model on the preprocessed dataset. Since these networks can learn hierarchical features from images automatically, they have proved to be particularly effective for image classification tasks. Some popular deep learning frameworks used in implementing CNNs are TensorFlow or PyTorch. A typical model architecture consists of multiple convolutional and pooling layers followed by fully connected layers for classification purposes. Techniques such as mini-batch gradient descent and backpropagation are employed during the training process where parameters are adjusted to reduce the classification error.

Step 4: Saving Files

After training a CNN model, its architecture and trained weights need to be saved into a format that is compatible with deployment. For instance, one may save models in TensorFlow using SavedModel format that encapsulates both weight and structure of the model in a portable manner.

Step 5: TensorFlow Serving

TensorFlow Serving is a flexible high-performance serving system for ML models. The second step involves deploying your trained model with TensorFlow Serving to make it accessible via network interfaces .In this case, TensorFlow Serving provides an easy way to serve your model over HTTP so clients can send.

Stage 6: FastAPI

The modern API building tool in Python is FastAPI. In this step, you will use FastAPI to build web service that sits between clients and your TensorFlow Serving instance. This feature enables you to define endpoints for input data, preprocess where necessary, send requests to the TensorFlow Serving instance and return model predictions to the client. FastAPI allows you to define API endpoints corresponding to different functionalities, such as receiving image data from clients, preprocessing it if necessary, forwarding requests to the TensorFlow Serving instance, and returning the model predictions back to the clients.

Step 7: Web Service

Finally, take your FastAPI-based web service to a server that can be accessed through the internet by clients. Clients interact with the web service in terms of HTTP requests sent through defined endpoints along with input data (e.g., images of plant leaves) and predictions about disease presence returned back. Without having to install or maintain any specialized software on their devices, this web service allows users to easily access your plant disease detection model.

4.2 DFDs & UML diagrams

DATA FLOW DIAGRAM

The DFDs are representative models that exhibit how data transfer happens from the input to output. In the Level 0 DFD, users submit images of plant leaves. In response, the system detects and identifies plant leaf diseases.

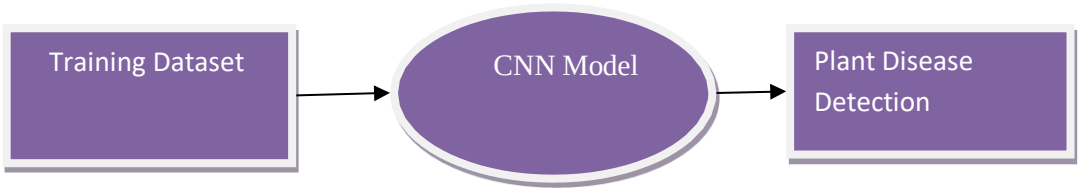


Fig. 4.2 DFD level 0 for Plant Disease Detection

Figure shows DFD Level 1 where CNN model takes an image from training dataset and predicts the kind of disease leaf.

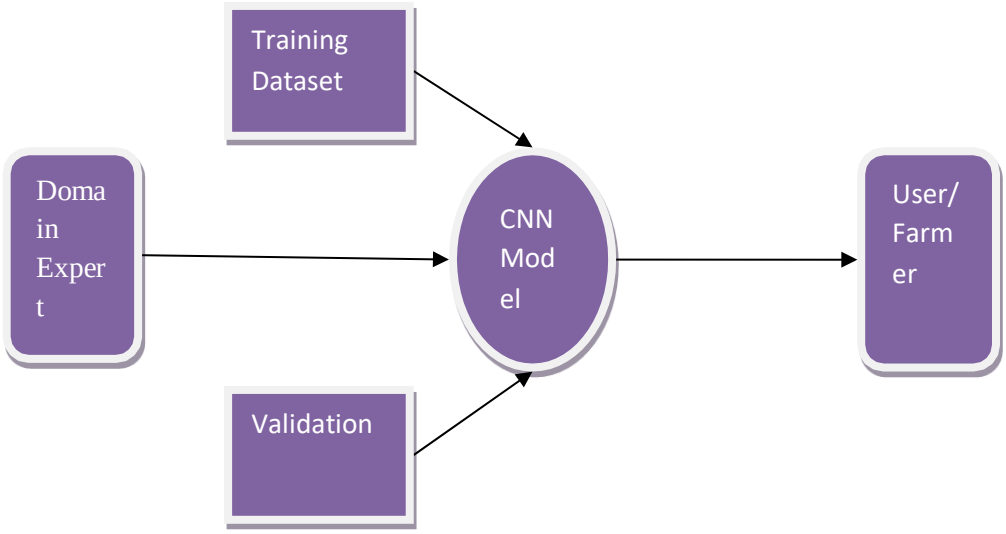


Fig. 4.3 DFD level 1 for Plant Disease Detection

DFD Level 2 is one step deeper than parts of 1-level DFD. It may be used to represent or capture specific detail about how a system functions. The DFD makes it possible to identify those actors that interact with a system, as well as designating the perimeter between our system and its environment. The latter also represents in brief on this diagram the Plant Disease website.

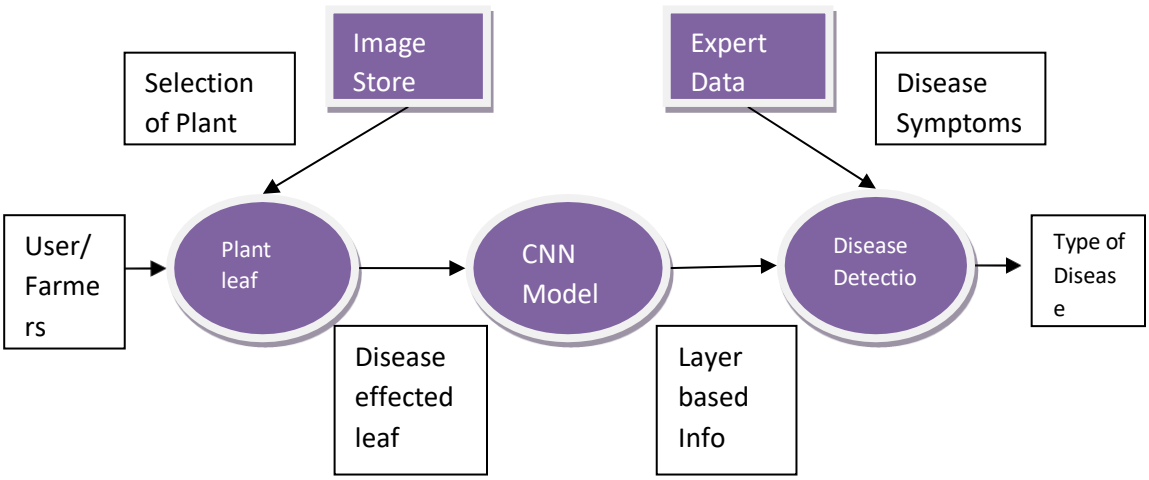


Fig. 4.4 DFD level 2 for Plant Disease Detection

The data flow chart presents a complete process of plant leaf analysis, ensuring a smooth interaction between users or farmers and the system. The first step is for users to upload images of their plant leaves into an image store and specify the type of plant they want to be analyzed. These photos are then sent to the Plant Leaf Analysis module that utilizes Convolutional Neural Networks (CNNs). Layered information that it collects helps CNN model to detect diseases existing in leaves. Afterwards, this disease detection output is sent to the Disease Detection component where it cross-refers with expected data containing disease symptoms. As a result, correct identification of disease types helps provide comprehensive understanding about disorders affecting plants. Users now have insights into what kind of disease their crops may have through these interconnected data flows could assist them in getting early indications for appropriate control measures.

Use Case Diagram

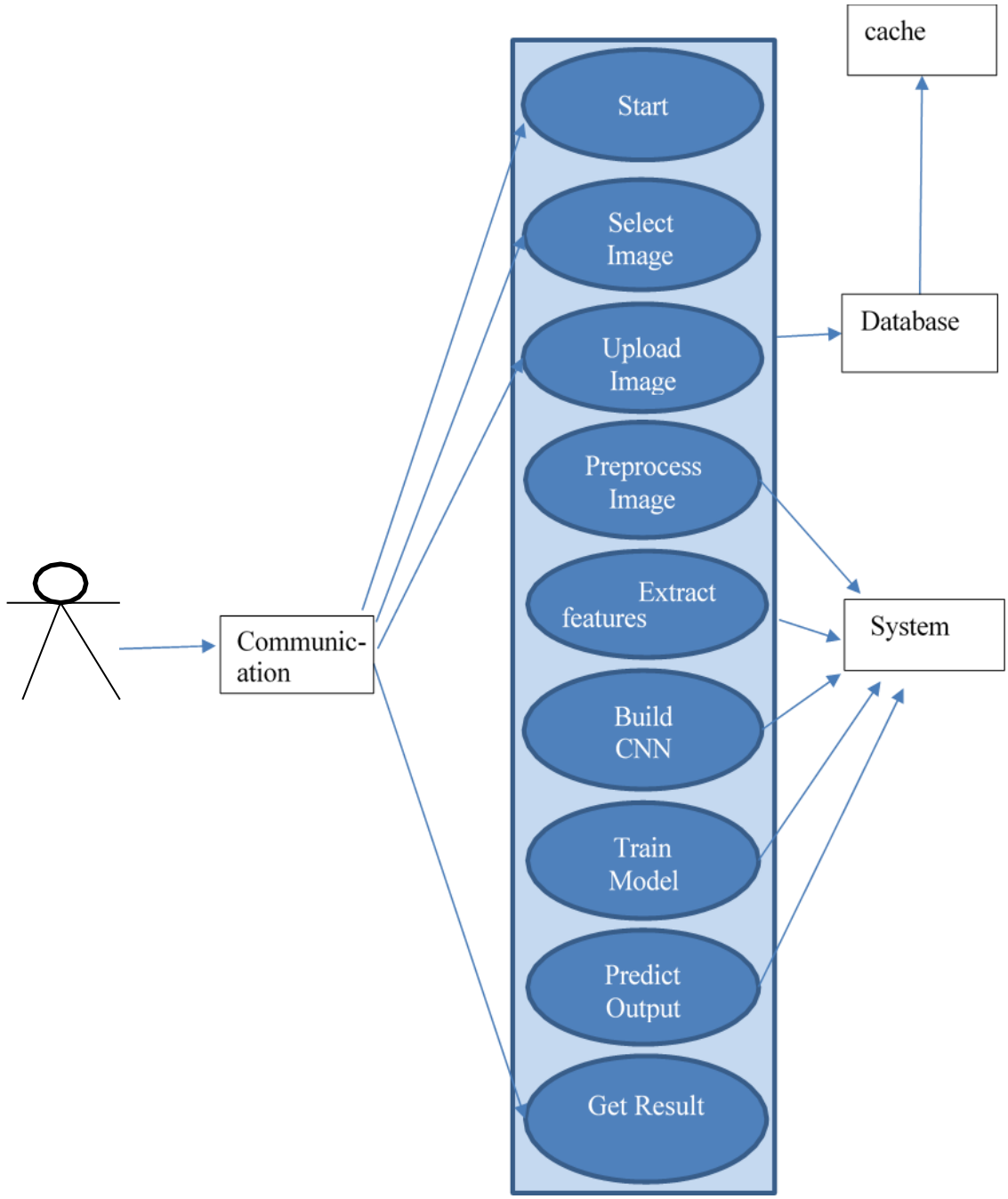


Fig. 4.5 Use Case Diagram

A UML Use Case Diagram is a diagram which presents actors of a system and use cases of the same system.

A use case represents function or group of functions from actor's perspective. Actors are those entities external to the system that interact with it through provided functionality during modeling process. These may include human users, other hardware devices or software systems. Ellipses represent use cases and dashed lines can be used to connect them to specific actors (and show which actor initiated the use case).

The use case diagram represents the working mechanism of a comprehensive plant disease detection system. At its core, there is a vertical order of processes that begins with the originating point called "Start". Thereafter, users interact with the system through such actions as "Select" (where they pick an image) or "Upload Image." Preprocessing is performed on the uploaded images following feature extraction and construction of a Convolutional Neural Network (CNN) for model training. Once this is done, this system can predict which type of plant disease it is based on the input image. At "Get Result", users will be able to obtain prediction results.

Hence, the communication medium becomes the bridge between web user inputs and system outputs. On the right side of this diagram there's an interconnected database linked to a cache for effective data storage and retrieval purposes. The system box situated at right hand side of this picture arranges inter-processes so that tasks are carried out seamlessly from image preprocessing up to illness prognosis.

Sequential Diagram

The previous diagram above illustrates how a simplified way to diagnose illness through machine learning can be done. There are tasks done by the algorithm and each is succeeded by another.

Picture Upload: The user uploads an image of the plant for K- users.

Resize Image: To enhance performance and consistency, the algorithm changes the size of the image and normalizes pixel values.

Segmentation: The algorithm uses segmentation techniques to divide various parts of the plant (e.g. stem leaves, flowers).

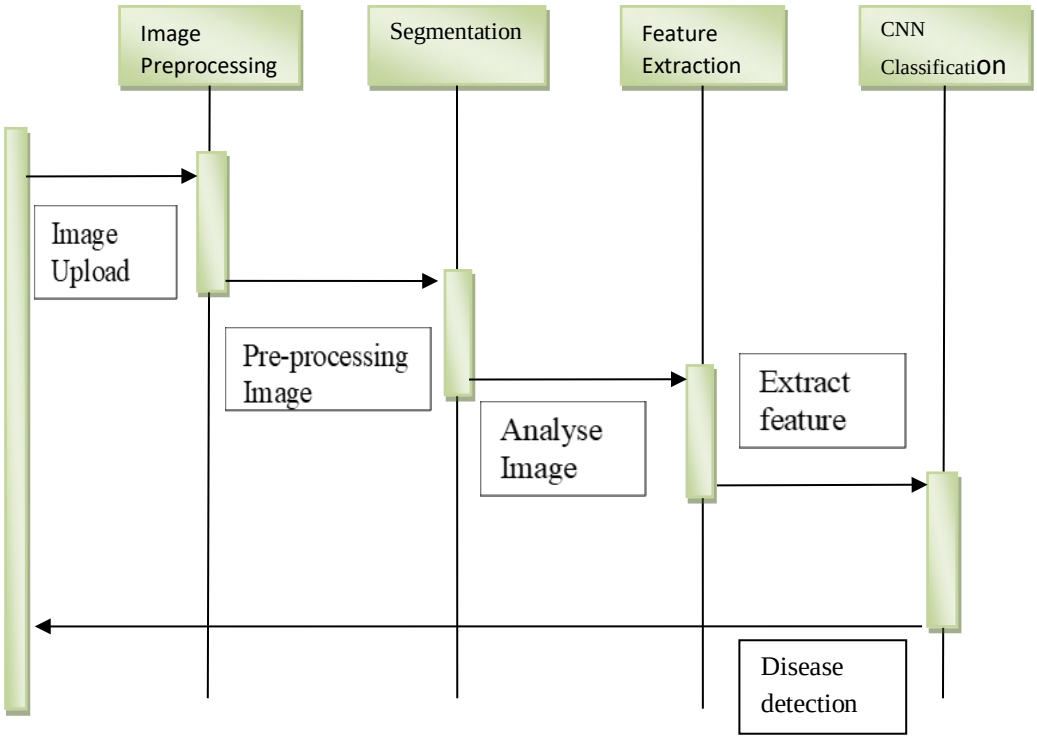


Fig. 4.6 Sequential Diagram

Pre-process Image: Every separated part of the plant is further pre-processed to improve efficiency in feature extraction.

Find Features: From these pre-processed images, our algorithm identifies distinctive features or patterns that will be inputs to CNN model.

Image Analyze: Extracted features are evaluated using a dataset that consists of both diseased and healthy K-user plants images and classify it as sick or healthy with help from CNN model.

Convolutional Neural Network (CNN) Classification: It's this framework which takes its features first before classifying with received training on how to do it eventually.

Features Extraction: This will assist in arriving at a final classification decision by extracting helpful characteristics from standardized images.

Diseased or Healthy: A prediction is produced by algorithm, indicating whether the K-user plant is healthy or diseased base on the input image.

4.3 Algorithm

Convolutional Neural Networks (CNN). This is the architecture as shown in figure above. As you can see the object itself may have different sizes. To solve this problem, an image pyramid is built by resizing the picture. The idea here is that we will resize the image at multiple scales and hope for one of these resized images to have the object entirely within our chosen window size (is completely contained inside our desired window size). Often, image is down sampled (size reduced) till certain condition most commonly a minimum size is reached. On each of these images, a fixed size window detector is run. Pyramids like these may have up t sixty four levels for instance. Then all these windows are fed into some classifier so as to detect whether there exists an object or not in them. Our aim here was to solve the issue of both size and location.

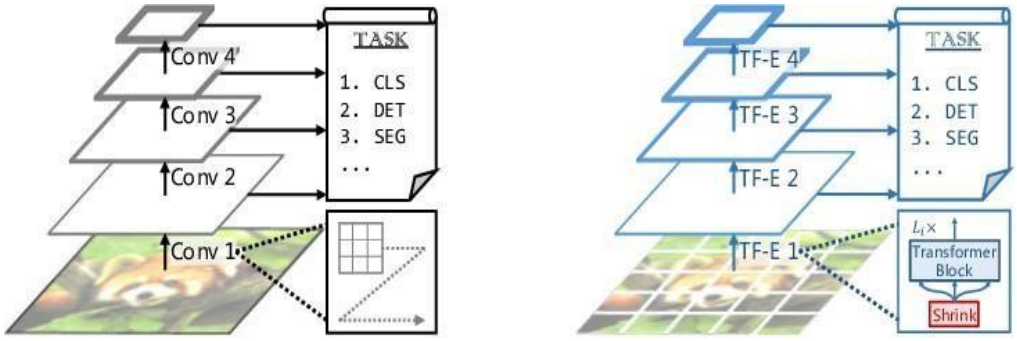


Fig 4.7. Image precision

Another challenge was that: it is necessary to have fixed sized input for fully connected layers of CNN so, SPP brings another trick here too! Instead of max-pooling traditionally used, it applies spatial pooling after the last convolutional layer. SPP layer divides a region of any arbitrary size into a constant number of bins and max pool is performed on each of the bins. Because of this, we have vector whose size never changes as indicated in the figure below.

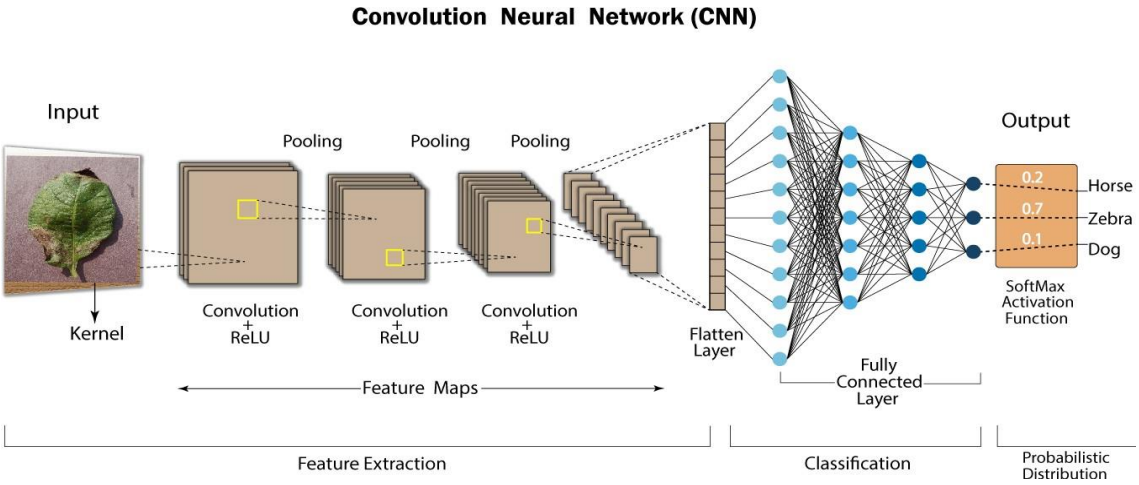


Fig 4.8. CNN Layer

CNN brings in the concept of anchor boxes to solve the problem of varying scale and aspect ratio. The original paper use 3 kinds of anchor boxes for scale 128x 128, 256×256 and 512×512 at each location. It also uses three aspect ratios for which they are: 1:1, 2:1, and 1:2. So it implies that there are nine boxes on each location over which RPN predicts background or foreground probability. We regress bounding box to improve anchor boxes at each location. RPN therefore gives different sizes bounding boxes with corresponding probabilities for each class representation. Applying CNN can necessitate further passing these different sized bounding boxes. This is why CNN has been one of the most accurate object detection algorithms developed so far. Here are some quick comparisons among various versions of CNN.

Input Layers: This is the layer we use to feed data into our model. In this layer, the number of neurons is determined by the total number of features in the data (number of pixels for images).

Hidden Layer: The hidden layer is where Input layer gives its input. Depending on model and data size many hidden layers can be present. Different numbers of neurons may be contained in every hidden layer which are generally more than that of features. Each new layer's output is computed by multiplying matrices through previous output with learnable weights per each, then adding learnable biases and an activation function that makes it nonlinear.

Output Layer: Consequently, it takes a logistic function such as sigmoid or softmax which changes each class's output to probability score for each class from a hidden layer output.

4.4 Sample Data

To use in this project, publicly accessible VILLAGE PLANT dataset will be employed. It has a total of 10k annotated images which contain 16 different types of objects with 25k objects annotations (.jpg format). These are images that were downloaded from Kaggle. This set was made in the year 2019. There are a total of several thousands of disease classes available in this dataset all related to plants with high accurate images.

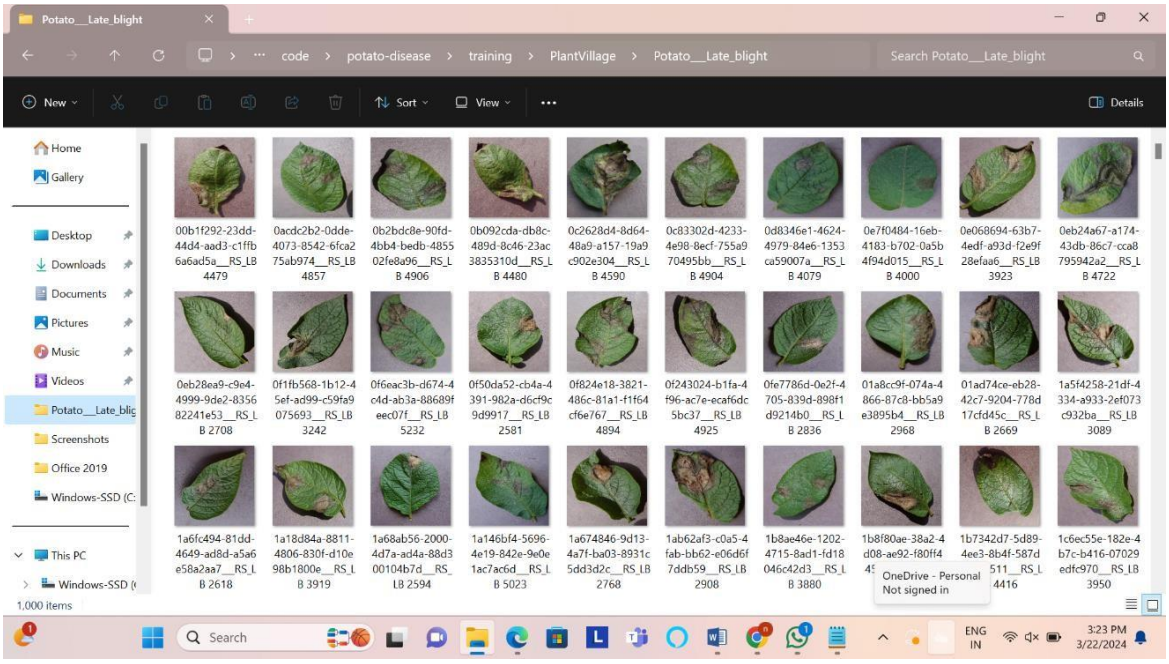


Fig.4.9 Sample Images

5. CODING

5.1 Pseudo Code

```
import tensorflow as tf
from tensorflow.keras import models, layers
import matplotlib.pyplot as plt

IMAGE_SIZE=256
BATCH_SIZE=32
CHANNELS=3
EPOCHS=10

dataset = tf.keras.preprocessing.image_dataset_from_directory(
    "PlantVillage",
    shuffle=True,
    image_size=(IMAGE_SIZE,IMAGE_SIZE),
    batch_size=BATCH_SIZE
)
class_names=dataset.class_names
class_names
len(dataset)

for image_batch, label_batch in dataset.take(1):
   print(image_batch.shape)
   print(label_batch.numpy())

for image_batch, label_batch in dataset.take(1):
   print(image_batch[0])
```

```
for image_batch, label_batch in dataset.take(1):
    print(image_batch[0].numpy())
for image_batch, label_batch in dataset.take(1):
    print(image_batch[0].shape)
for image_batch, label_batch in dataset.take(1):
    plt.imshow(image_batch[0].numpy().astype("uint8"))
    plt.title(class_names[label_batch[0]])
    plt.axis("off")

plt.figure(figsize=(10,10))
for image_batch, label_batch in dataset.take(1):
    for i in range(12):
        ax=plt.subplot(3,4,i+1)
        plt.imshow(image_batch[i].numpy().astype("uint8"))
        plt.title(class_names[label_batch[i]])
        plt.axis("off")
len(dataset)

#spliting
train_size=0.8
len(dataset)*train_size
train_ds=dataset.take(54) #[:54]
len(train_ds)
test_ds=dataset.skip(54)
len(test_ds)
val_size=0.1
len(dataset)*val_size
val_ds=test_ds.take(6)
len(val_ds)

test_ds=test_ds.skip(6)
```

```
len(test_ds)

def get_dataset_partitions_tf(ds, train_split=0.8, val_split=0.1, test_split=0.1,
shuffle=True, shuffle_size=10000): #these function can manage the data
    ds_size=len(ds)
    if shuffle:
        ds=ds.shuffle(shuffle_size,seed=12)
    train_size=int(train_split*ds_size)
    val_size=int(val_split*ds_size)
    train_ds=ds.take(train_size)
    val_ds=ds.skip(train_size).take(val_size)
    test_ds=ds.skip(train_size).skip(val_size)
    return train_ds,val_ds,test_ds

train_ds, val_ds, test_ds=get_dataset_partitions_tf(dataset)
len(train_ds)
len(val_ds)
len(test_ds)

train_ds=train_ds.cache().shuffle(1000).prefetch(buffer_size=tf.data.AUTOTUNE)#prefe
tch used for if u r gpu is busy it will another setup of batch from your disk
val_ds=val_ds.cache().shuffle(1000).prefetch(buffer_size=tf.data.AUTOTUNE)
test_ds=test_ds.cache().shuffle(1000).prefetch(buffer_size=tf.data.AUTOTUNE)

#preprocessing
resize_and_rescale=tf.keras.Sequential([
    layers.experimental.preprocessing.Resizing(IMAGE_SIZE,IMAGE_SIZE),
    layers.experimental.preprocessing.Rescaling(1.0/255)
])
```

```
data_augmentation=tf.keras.Sequential([
    layers.experimental.preprocessing.RandomFlip("horizontal_and_vertical"),
    layers.experimental.preprocessing.RandomRotation(0.2),
])
input_shape=(BATCH_SIZE,IMAGE_SIZE,IMAGE_SIZE, CHANNELS)
n_classes=3

model=models.Sequential([
    resize_and_rescale,
    data_augmentation,
    layers.Conv2D(32,(3,3),activation='relu',input_shape=input_shape),
    layers.MaxPooling2D((2,2)),
    layers.Conv2D(64,kernel_size=(3,3),activation='relu'),
    layers.MaxPooling2D((2,2)),
    layers.Conv2D(64,kernel_size=(3,3),activation='relu'),
    layers.MaxPooling2D((2,2)),
    layers.Conv2D(64,(3,3),activation='relu'),
    layers.MaxPooling2D((2,2)),
    layers.Conv2D(64,(3,3),activation='relu'),
    layers.MaxPooling2D((2,2)),
    layers.Conv2D(64,(3,3),activation='relu'),
    layers.MaxPooling2D((2,2)),
    layers.Flatten(),
    layers.Dense(64,activation='relu'), #dense layer of 64 neurans
    layers.Dense(n_classes,activation='softmax'),  #softmax activation function used for
normalized the propablity of a classes
])
model.build(input_shape=input_shape)
model.summary()

model.compile(
```

```
    optimizer='adam',
    loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=False),
    metrics=['accuracy']
)
history=model.fit(
    train_ds,
    epochs=EPOCHS,
    batch_size=BATCH_SIZE,
    verbose=1,
    validation_data=val_ds
)
scores=model.evaluate(test_ds)
scores
history.params
history.history.keys()
history.history['accuracy']
acc=history.history['accuracy']
val_acc=history.history['val_accuracy']
loss=history.history['loss']
val_loss=history.history['val_loss']

plt.figure(figsize=(8, 8))
plt.subplot(1, 2, 1)
plt.plot(range(EPOCHS), acc, label='Training Accuracy')
plt.plot(range(EPOCHS), val_acc, label='Validation Accuracy')
plt.legend(loc='lower right')
plt.title('Training and Validation Accuracy')
plt.subplot(1, 2, 2)
plt.plot(range(EPOCHS), loss, label='Training Loss')
plt.plot(range(EPOCHS), val_loss, label='Validation Loss')
plt.legend(loc='upper right')
```

```
plt.title('Training and Validation Loss')
plt.show()

for images_batch, labels_batch in test_ds.take(1):
    # print(images_batch[0])
    plt.imshow(images_batch[0].numpy().astype('uint8')) #actuall image

for images_batch, labels_batch in test_ds.take(1):
    print(images_batch[0].numpy().astype('uint8')) #three dimensional array rgb

import numpy as np
for images_batch, labels_batch in test_ds.take(1):
    first_image=images_batch[0].numpy().astype('uint8')
    first_label=labels_batch[0].numpy()
    print("first image to predict")
    plt.imshow(first_image)
   # print("first image's actual label:", class_names[first_label])#first_label
    print("actual label:",class_names[first_label])
    batch_prediction=model.predict(images_batch)
    print("preidct label:",class_names[np.argmax(batch_prediction[0])])

def predict(model, img):
    img_array=tf.keras.preprocessing.image.img_to_array(images[i].numpy())
    img_array=tf.expand_dims(img_array,0) #create a batch
    predictions=model.predict(img_array)
    predicted_class=class_names[np.argmax(predictions[0])]
    confidence=round(100*(np.max(predictions[0])),2)
    return predicted_class, confidence
plt.figure(figsize=(15,15))
for images, labels in test_ds.take(1):
    for i in range(9):
```

```
        ax=plt.subplot(3,3,i+1)
        plt.imshow(images[i].numpy().astype("uint8"))
        predicted_class, confidence =predict(model, images[i].numpy())
        actual_class= class_names[labels[i]]
        plt.title(f"Actual: {actual_class},\n Predicted: {predicted_class}.\n Confidence:
{confidence}%"
        plt.axis('off')

import os
model_version=max([int(i) for i in os.listdir("../saved_models") + [0]])+1
model.save(f"../saved_models/{model_version}")
```

Main.py

```
from fastapi import FastAPI,File,UploadFile
import numpy as np
from io import BytesIO
import uvicorn
from PIL import Image
import tensorflow as tf
from fastapi.middleware.cors import CORSMiddleware

app = FastAPI()

origins = [
    "http://localhost",
    "http://localhost:3000",
]
app.add_middleware(
    CORSMiddleware,
    allow_origins=origins,
```

```
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

MODEL = tf.keras.models.load_model("../saved_models/1")
CLASS_NAMES = ["Early Blight", "Late Blight", "Healthy"]



@app.get("/ping")
async def ping():
    return "Hello, I am alive"

def read_file_as_image(data) -> np.ndarray:
    image = np.array(Image.open(BytesIO(data)))
    return image

@app.post("/predict")
async def predict(
        file: UploadFile = File(...)
):
    image = read_file_as_image(await file.read())
    img_batch = np.expand_dims(image, 0)

    predictions = MODEL.predict(img_batch)

    predicted_class = CLASS_NAMES[np.argmax(predictions[0])]
    confidence = np.max(predictions[0])
    return {
        'class': predicted_class,
        'confidence': float(confidence)
```

```
    }

if __name__ == "__main__":
    uvicorn.run(app, host='localhost', port=8000)
```

index.html

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="utf-8" />
    <link rel="icon" href="%PUBLIC_URL%/cblogo.PNG" />
    <meta name="viewport" content="width=device-width, initial-scale=1" />
    <meta name="theme-color" content="#000000" />
    <meta
      name="description"
      content="Web site created using create-react-app"
    />
    <link rel="apple-touch-icon" href="%PUBLIC_URL%/logo192.png" />

    <link rel="manifest" href="%PUBLIC_URL%/manifest.json" />
    <title>Plant Disease</title>
  </head>
  <body>
    <noscript>You need to enable JavaScript to run this app.</noscript>
    <div id="root"></div>
  </body>
</html>
```

App.js

```
import { ImageUpload } from "./home";

function App() {
  return <ImageUpload />;
}

export default App;
```

home.js

```
import { useState, useEffect } from "react";
import { makeStyles, withStyles } from "@material-ui/core/styles";
import AppBar from "@material-ui/core/AppBar";
import Toolbar from "@material-ui/core/Toolbar";
import Typography from "@material-ui/core/Typography";
import Avatar from "@material-ui/core/Avatar";
import Container from "@material-ui/core/Container";
import React from "react";
import Card from "@material-ui/core/Card";
import CardContent from "@material-ui/core/CardContent";
import { Paper, CardActionArea, CardMedia, Grid, TableContainer, Table, TableBody,
TableHead, TableRow, TableCell, Button, CircularProgress } from "@material-ui/core";
import cblogo from "./cblogo.PNG";
import image from "./bg.png";
import { DropzoneArea } from 'material-ui-dropzone';
import { common } from '@material-ui/core/colors';
import Clear from '@material-ui/icons/Clear';
```

```
const ColorButton = withStyles((theme) => ({
  root: {
    color: theme.palette.getContrastText(common.white),
    backgroundColor: common.white,
    '&:hover': {
      backgroundColor: '#ffffff7a',
    },
  },
}))(Button);
const axios = require("axios").default;

const useStyles = makeStyles((theme) => ({
  grow: {
    flexGrow: 1,
  },
  clearButton: {
    width: "-webkit-fill-available",
    borderRadius: "15px",
    padding: "15px 22px",
    color: "#000000a6",
    fontSize: "20px",
    fontWeight: 900,
  },
  root: {
    maxWidth: 345,
    flexGrow: 1,
  },
  media: {
    height: 400,
  },
```

```
paper: {
  padding: theme.spacing(2),
  margin: 'auto',
  maxWidth: 500,
},
gridContainer: {
  justifyContent: "center",
  padding: "4em 1em 0 1em",
},
mainContainer: {
  backgroundImage: `url(${image})`,
  backgroundRepeat: 'no-repeat',
  backgroundPosition: 'center',
  backgroundSize: 'cover',
  height: "93vh",
  marginTop: "8px",
},
imageCard: {
  margin: "auto",
  maxWidth: 400,
  height: 500,
  backgroundColor: 'transparent',
  boxShadow: '0px 9px 70px 0px rgb(0 0 0 / 30%) !important',
  borderRadius: '15px',
},
imageCardEmpty: {
  height: 'auto',
},
noImage: {
  margin: "auto",
  width: 400,
```

```
  height: "400 !important",
},
input: {
  display: 'none',
},
uploadIcon: {
  background: 'white',
},
tableContainer: {
  backgroundColor: 'transparent !important',
  boxShadow: 'none !important',
},
table: {
  backgroundColor: 'transparent !important',
},
tableHead: {
  backgroundColor: 'transparent !important',
},
tableRow: {
  backgroundColor: 'transparent !important',
},
tableCell: {
  fontSize: '22px',
  backgroundColor: 'transparent !important',
  borderColor: 'transparent !important',
  color: '#000000a6 !important',
  fontWeight: 'bolder',
  padding: '1px 24px 1px 16px',
},
tableCell1: {
  fontSize: '14px',
```

```
  backgroundColor: 'transparent !important',
  borderColor: 'transparent !important',
  color: '#000000a6 !important',
  fontWeight: 'bolder',
  padding: '1px 24px 1px 16px',
},
tableBody: {
  backgroundColor: 'transparent !important',
},
text: {
  color: 'white !important',
  textAlign: 'center',
},
buttonGrid: {
  maxWidth: "416px",
  width: "100%",
},
detail: {
  backgroundColor: 'white',
  display: 'flex',
  justifyContent: 'center',
  flexDirection: 'column',
  alignItems: 'center',
},
appbar: {
  background: '#6abeb1',
  boxShadow: 'none',
  color: 'white'
},
loader: {
  color: '#be6a77 !important',
```

```
  }
}));
export const ImageUpload = () => {
  const classes = useStyles();
  const [selectedFile, setSelectedFile] = useState();
  const [preview, setPreview] = useState();
  const [data, setData] = useState();
  const [image, setImage] = useState(false);
  const [isLoading, setIsloading] = useState(false);
  let confidence = 0;

  const sendFile = async () => {
    if (image) {
      let formData = new FormData();
      formData.append("file", selectedFile);
      let res = await axios({
        method: "post",
        url: process.env.REACT_APP_API_URL,
        data: formData,
      });
      if (res.status === 200) {
        setData(res.data);
      }
      setIsloading(false);
    }
  }

  const clearData = () => {
    setData(null);
    setImage(false);
    setSelectedFile(null);
```

```
  setPreview(null);
};

useEffect(() => {
  if (!selectedFile) {
    setPreview(undefined);
    return;
  }
  const objectUrl = URL.createObjectURL(selectedFile);
  setPreview(objectUrl);
}, [selectedFile]);

useEffect(() => {
  if (!preview) {
  return;
  }
  setIsloading(true);
  sendFile();
}, [preview]);

const onSelectFile = (files) => {
  if (!files || files.length === 0) {
  setSelectedFile(undefined);
  setImage(false);
  setData(undefined);
    return;
  }
  setSelectedFile(files[0]);
  setData(undefined);
  setImage(true);
};
```

```
  if (data) {
    confidence = (parseFloat(data.confidence) * 100).toFixed(2);
  }

  return (
    <React.Fragment>
      <AppBar position="static" className={classes.appbar}>
        <Toolbar>
          <Typography className={classes.title} variant="h6" noWrap>
            Plant Disease
          </Typography>
          <div className={classes.grow} />
          <Avatar src={cblogo}></Avatar>
        </Toolbar>
      </AppBar>
      <Container maxWidth={false} className={classes.mainContainer}
disableGutters={true}>
        <Grid
          className={classes.gridContainer}
          container
          direction="row"
          justifyContent="center"
          alignItems="center"
          spacing={2}
        >
          <Grid item xs={12}>
            <Card className={`${classes.imageCard} ${!image ? classes.imageCardEmpty :
"}`}>
              {image && <CardActionArea>
                <CardMedia
```

```
            className={classes.media}
            image={preview}
            component="image"
            title="Contemplative Reptile"
          />
        </CardActionArea>
        }
        {!image && <CardContent className={classes.content}>
          <DropzoneArea
            acceptedFiles={['image/*']}
            dropzoneText={"Drag and drop an image of a potato plant leaf to process"}
            onChange={onSelectFile}
          />
        </CardContent>}
        {data && <CardContent className={classes.detail}>
          <TableContainer component={Paper} className={classes.tableContainer}>
            <Table className={classes.table} size="small" aria-label="simple table">
              <TableHead className={classes.tableHead}>
                <TableRow className={classes.tableRow}>
                  <TableCell className={classes.tableCell1}>Label:</TableCell>
                  <TableCell align="right"
className={classes.tableCell1}>Confidence:</TableCell>
                </TableRow>
              </TableHead>
              <TableBody className={classes.tableBody}>
                <TableRow className={classes.tableRow}>
                  <TableCell component="th" scope="row"
className={classes.tableCell}>
                    {data.class}
                  </TableCell>
```

```
                    <TableCell align="right"
className={classes.tableCell}>{confidence}%</TableCell>
                  </TableRow>
                </TableBody>
              </Table>
            </TableContainer>
          </CardContent>}
          {isLoading && <CardContent className={classes.detail}>
            <CircularProgress color="secondary" className={classes.loader} />
            <Typography className={classes.title} variant="h6" noWrap>
              Processing
            </Typography>
          </CardContent>}
        </Card>
      </Grid>
      {data &&
        <Grid item className={classes.buttonGrid} >

          <ColorButton variant="contained" className={classes.clearButton}
color="primary" component="span" size="large" onClick={clearData} startIcon={<Clear
fontSize="large" />}>
            Clear
          </ColorButton>
        </Grid>}
    </Grid >
  </Container >
 </React.Fragment >
 );
};
```

6. IMPLEMENTATION & RESULTS

6.1 Explanation of Key functions

6.1.1 Model functions

It is particularly effective to use CNNs for image classification tasks as such are well-suited for detecting diseases in plants by examining leaves or other parts of the plant.

Convolutional Layer: It is the main operation in a CNN. It employs convolutional filters on the input image to extract various elements such as edges, textures, or patterns. Each filter learns how to detect specific features in an image.

Activation Function (e.g., ReLU): After every convolutional operation, an activation function is applied element-wise to bring nonlinearity into the network. Rectified Linear Unit (ReLU) is a common non-linear activation function which outputs zero if it receives negative inputs and otherwise outputs its inputs directly.

Pooling Layer (e.g., MaxPooling): Pooling layers down sample the convolutional layers while retaining most important information. For example, MaxPooling selects maximum value within a defined window thus reducing feature maps dimensionality.

Flattening: The feature maps are flattened into one-dimensional vector before passing them through fully connected layers. This operation transforms output from convolution/pooling layers so that it could be fed into dense layers.

Dense Layer (Fully Connected): These layers take flattened features and separate them according to their classification. In a fully connected layer, every neuron is linked with all the neurons in the previous layer. They are useful for capturing intricate data patterns.

Dropout: A regularization technique that prevents neural networks from becoming overfitted is known as dropout. The technique involves randomly zeroing out some of the input units during training which ensures that the network does not rely too heavily on any specific set of features.

Softmax Activation: Often found in multi-class classification problems, this activation function is used at the end of the network. Softmax transforms raw output scores into probabilities that can be interpreted as class likelihoods.

Loss Function (e.g., Categorical Cross-Entropy): This specifies how much difference there is between predicted output and actual target labels. For example, when dealing with classification tasks, categorical cross-entropy measures similarity between predicted class probabilities and true class labels.

Optimizer (e.g., Adam): The optimizer changes the weights of the network with respect to derivatives of discharge function obtained with respect to weights. A popular optimization algorithm which combines ideas from RMSProp and Momentum called Adam opens a way for efficient optimization of deep learning models.

Learning Rate: This hyperparameter determines how fast the weights will be modified during training. One should fine-tune this parameter in order to achieve an effective convergence over time.

6.1.2 Interface functions

Home Page:

Input:

Advertisements user interaction on (slides),

Plant type options to select (e.g., tomato, pepper bell, potato).

Output:

Visually supported plant alternatives.

Giving a visual option for selecting the species of plants.

Classification Page:

Input:

An input image of a plant leaf suspected to have disease that is uploaded or captured.

Output:

Display on the page, an uploaded/captured image depicted visually.

Predict Disease:

Input:

Click on 'predict' button or other similar interface element as available.

Output:

On the screen, predicted disease label with its confidence scores.

Show Classification Results:

Input:

Outcome of the prediction process.

Output:

Classification results showing detected disease together with other information such as certainty level and recommended actions.

Clear Option:

Input:

User decides to return to the Home Page or previous page.

Output:

Navigating back to the Home Page in order to select another disease or do a new classification.

About Page:

Input: Clicking on the About page.

Output: How plant diseases are classified and detected, inclusive of particulars concerning sampling techniques, live streaming and provided services.

Services Page:

Input: Navigation to the Services page.

Output: Full information about offered services i.e. current offerings and future plans.

Organic Page:

Input: Selection of the Organic page.

Output: Information regarding traditional organic farming methods.

Blog Page:

Input: Accessing the Blog page.

Output: Whereby various blogs on different aspects of farming can be found here too, as well as photos that will help you understand some concepts better!

Details Page:

Input: Navigation to the Details page.

Output: In other words, it is explained in details what a crop disease is; how it manifests itself and also how it is diagnosed and treated?

Gallery Page:

Input: Clicking on the Gallery page.

Output: Plant diseases photos, agriculture-related pictures etc., are presented here

Contact Page:

Input: Accessing the Contact page.

Output: Having diverse ways through which one may reach out for support regarding this project such as e-mail address, phone number, physical location –address- on google maps.

Feedback Form:

Input: Filling out the feedback form.

Output: Collecting user feedback on the company, its functionalities, and user experience.

6.2 Method of Implementation

Our experiments were evaluated on a GPU NVIDIA workstation as the baseline system. The operating environment was Windows 11, with GDDR5 graphic memory, Core i5 10th generation and 8 GB RAM. Software implementation was done by use of the Anaconda3. Keras, TENSORFLOW, Numpy, and matplotlib libraries. The implantation of this software is performed via the Anaconda3 framework in combination with its accompanying libraries which are Keras, TENSORFLOW, Numpy, and matplotlib respectively.tf serving simple libraries are explicitly built to do deep learning implementations with less memory or faster execution. Theano backend of both these libraries are designed and implemented by NVIDIA itself. Fastapi supports academic projects as well as commercial ones and works with Linux, Windows, Mac OS, iOS, Python, Java interfaces for Android.

The fast api can be used equally well for scientific research projects as well as for environmental protection tasks such as regulations compliance among others in addition to commercial purposes.

For each experiment carried out in this study we have evaluated training accuracy and testing accuracy individually. Losses computed during test phase and train phase are calculated for each model. The plant village dataset is used to train models so that the transfer learning models for CNN can be learned quickly. Pre-trained models selected for our investigation included CNN which had been trained before using Plant Village dataset of 1/2 GB images image categories.

6.2.1 Description of Dataset

PlantVillage dataset is a publicly available dataset that has various plant disease categories. This dataset has 15 classes with 20645 pictures. For our experiment, we divided the data into train samples, test samples and validation samples. The PlantVillage dataset is an extensive collection of images intended to support research and development in plant pathology as well as agriculture. It has a wide range of plant diseases and healthy status that are useful in studying and understanding different plant ailments. This dataset contains a significant portion dedicated to pictures of healthy plants which act as controls when comparing with the diseased samples.

80% of the PlantVillage dataset was used for training the pre-trained models while the remaining 20% was reserved for validation and testing purposes. Furthermore, there were a total of 20645 plant class samples out of which 18032 were used as training samples,1376 as validation ones and 1654 as test data. These train-test-validation sets consist of all the fifteen classes of different plant diseases. The distributional details about these datasets are presented in Table:1

Disease Class	Total Samples	Training Samples	Testing Samples	Validation Samples
Pepper_bell_bacterial_spot	997	807	100	90
Pepper_bell_healthy	1478	1197	148	133
Potato_early_blight	1000	810	100	90
Potato_healthy	1000	810	100	90
Potato_late_blight	152	122	16	14
Tomato_bacterial_spot	2127	1722	213	192
Tomato_early_blight	1000	810	100	90
Tomato_healthy	1591	1546	191	172
Tomato_late_blight	1909	770	96	86
Tomato_leaf_mold	952	1433	178	160
Tomato_septoria_leaf_spot	1771	1357	168	151
Tomato_spider_mites_two Spotted_spider_mite	1676	1136	141	127
Tomato_target_spot	1404	4338	536	483
Tomato_mosaic_virus	373	307	38	34
Tomato_yellow_leaf_curl_virus	3209	1287	160	144
Total	20,639	18,446	2,285	2,056

Table.6.1 Details of image

6.2.2 Preprocessing And Augmentation

There were 15 classes in the dataset that contained 15 diseases and three kinds of crops. In our experiments, we used color images from the PlantVillage dataset because they are consistent with transfer learning models.

We downscaled the images to 256 × 256 pixels since we used different pre-trained network models for which input sizes varied. For CNN, input size is 224 × 224 × 3 (height, width, channel width) but for Inception V4 input shape of images should be 299 × 299 × 3 (height, width, channel width). The crop diseases under discussion differ in a huge data set of nearly around 20639 images which have been obtained from different image acquisition technologies such as kinect sensors or smart phones used by farmers in taking photographs like HD cameras and so on.

This however increases risk of overfitting due to large dataset. Consequently to mitigate this problem some methods such as data augmentation after preprocessing have been introduced as overfitting regularization techniques. Preprocessed images were rotated clockwise and anticlockwise, flipped horizontally and vertically and zoom intensity among others during augmentation process. The training did not involve duplication of images but rather augmentation with them. During training, the images were not duplicated but increased; therefore, instead of storing physical copies of the amplified images, they were temporarily used. This method of augmentation helps not only in preventing model loss and overfitting but also improves the robustness of the model as a result, when applying for differentiating between real life plant disease imagery by using these models it can do this with more accuracy.

6.2.3 Fine-Tuning of Hyperparameters in Pre-Trained Models

Transfer learning model has many advantages. It learns more quickly as compared to models created from scratch. Similarly, certain layers of the model can be frozen while its last layers are trained in order to provide better classification results. The initial stage involved performing some normalizations of the hyperparameters for various pre-trained models. This process is described more fully in Table 2.

Hyperparameters	Epochs
Dropout	0.5
Epochs	30
Activation	ReLU
Regularization	Batch normalization
Optimizer	Stochastic gradient descent
Learning rate	0.001
Output classes	38

Table.6.2 Details of Hyperparameters

6.2.4 Network Architecture Model

The choice of pre-trained network models was based on their suitability for plant disease classification task. For each network different filter sizes are employed to extract specific features from feature maps. Feature extraction hinges on filters.

Then, every filter shapes a unique image when stretched over inputs since its characteristic picks out distinctive values from feature maps. In our experiments, we used an actual pre-trained network model with actual convolution layers and actual filter sizes used by each network model.

6.2.5 Result Analysis

In this study, we applied advanced deep learning models with transfer learning approach to diagnose plant diseases. The train deep CNNs were pre-tuned on the ImageNet dataset and fine-tuned them using the Publicly available PlantVillage dataset. For each model, the experiment standardized with a specific set of parameters: a learning rate of 0.01, dropout rate of 0.5 and output classes of 15.

Performance analysis based on specificity, sensitivity as well as F1 score for different pretrained models. Achieving a classification accuracy of 98.27% is certainly an impressive result, indicating that your model is performing very well on the given task.

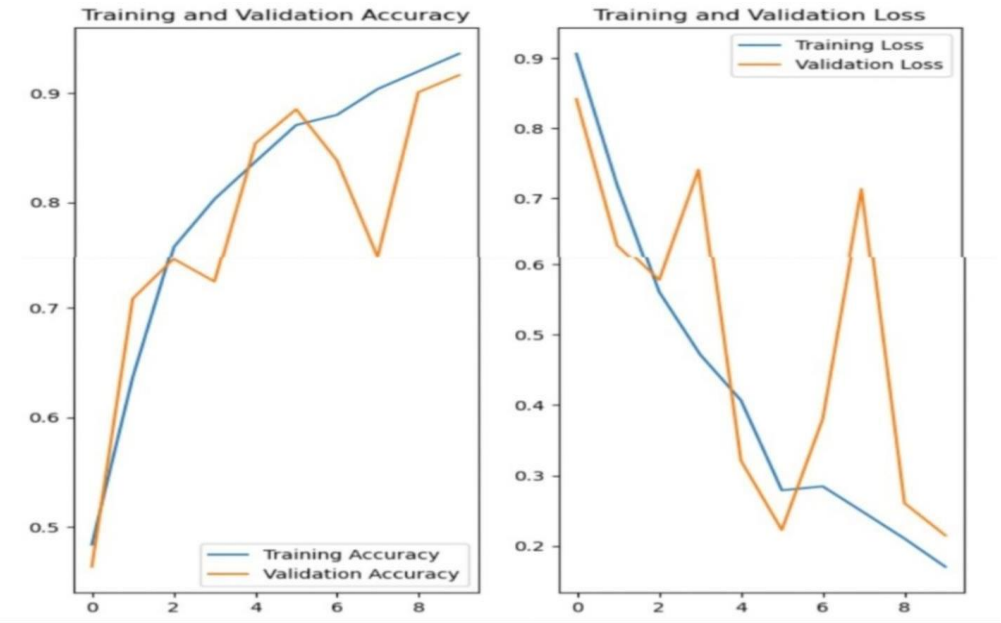


Fig. 6.1 Result graph

6.3 TECHNOLOGIES USED

6.3.1 Python

Fig.6.2 Python image

1. A widely used, general-purpose, high-level programming language is Python.

2. Object-Oriented and Procedural programming can be done using python

3. Compared to Java or C++, Python programmers have to type very little and the language's requirement for indentation makes it neat-looking at all times.

4. Python is being used in almost every tech-giant company Google, Amazon, Facebook, Instagram, Dropbox… etc

5. The biggest advantage of Python is its large library which can be used for:

- Learning machines
- GUI Applications
- Reatjs web frameworks
- Processing Images
- Scraping Web Pages
- Frameworks for testing
- Multimedia
- Text processing

6.3.2 Machine Learning

Machine Learning, which is the study that gives computers the ability to learn without being officially told what to do. It could be said that Machine Learning (ML) is one of the most exciting technology innovations in existence so far. The name itself means giving the computer a quality similar to human: Learning ability.

APPROACHES TO MACHINE LEARNING

An early classification of machine learning approaches separated them into three large categories according to the type of "signal" or feedback available to the learning system. They included:

Supervised learning: In this case, a teacher presents example inputs and their desired outputs to the computer with a hope that it will pick up some general rule connecting them.

Unsupervised learning: The learning algorithm receives no labels; hence it has to find its own structure in its patterns. The goal of unsupervised learning can either be finding hidden structures within data or feature extraction in preparation for other tasks.

Reinforcement learning: A computer program that is propelled by reinforcement learning would interact with a vibrant environment, which it ought to meet a particular goal (like driving or playing a game against an adversary).

Along its path through the problem space, the program receives feedback comparable to rewards that it strives to optimize. There are also alternative ways of thinking about this framework; other perspectives have been proposed which do not fit neatly into these three categories, and often times one machine learning system may use more than one of them at the same time.

6.3.3 Deep Learning

Deep learning is a subdivision of machine learning that revolves entirely on artificial neural networks since these as its name suggests are going to copy the human brain thus in essence, deep leaming is a mimicry of the human bram. Explicit programming is no longer required in deep learning. Deep learning has been around for quite some time now. Nowadays, it's being hyped up because we didn't have enough processing power and massive data back then. As processing power has grown exponentially over the last 20 years, so did deep leaming and machine learning enter the scene.

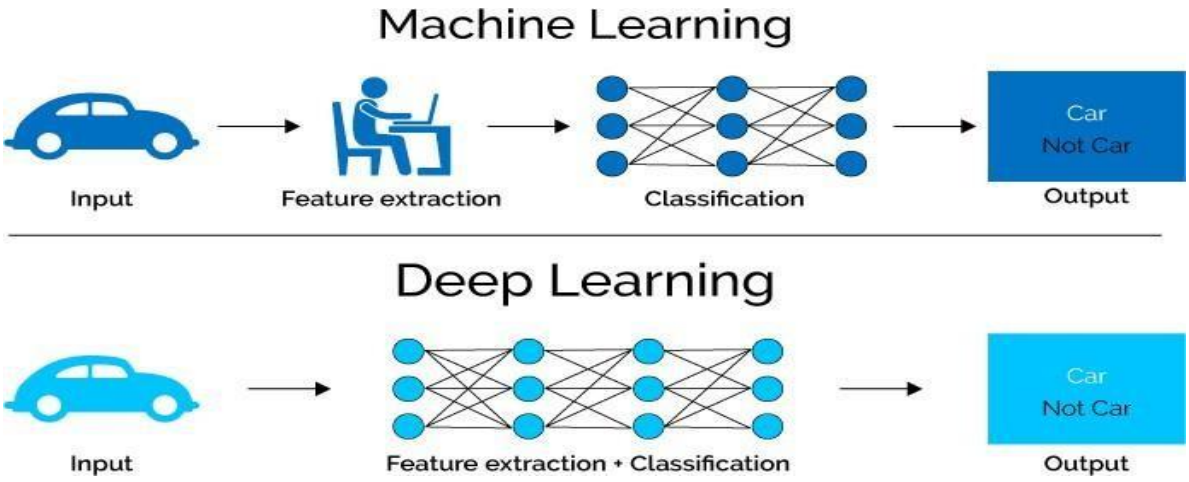


Fig.6.3 ML vs DL

Although modern deep learning models are mainly based on artificial neural networks such as Convolutional Neural Networks (CNN)s, they could also include propositional formulas or latent variables organized layer-wise in deep generative models that resemble nodes of deep belief networks or deep Boltzmann machines. Deep learning functions by having input data transformed into slightly more abstract and composite representations at each stage.

Networks analyzing images may use raw pixels as inputs; a first layer of representation will extract features that can be used to encode edges; The second layer may encode and compose orderliness of edges; the third layer might encode a nose and eyes, while the fourth layer might know that there is a face in the picture.

6.3.4.Data Science

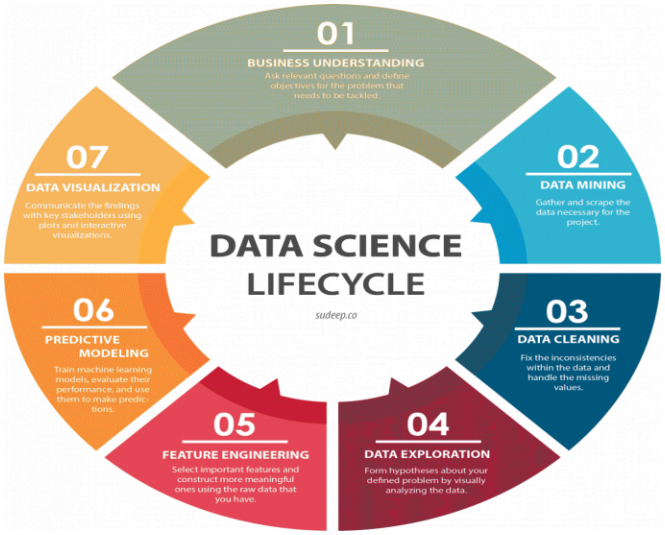


Fig.6.4 Data Science life cycle

The study of data is what data science is. Just as biological sciences are concerned with biology so too physical sciences deal with physical reactions Data is real, it has properties and these properties must be researched if the processes will always be examined Data Science entails both information and signals. It is a never-ending process, not an event that takes place in single time frame. And you see this all over the world where people use data to know everything. That's when data becomes a narrative for you; just make a story out of it. Use stories, therefore, to generate insights. These insights can help someone decide on strategies for their company or organization. This also implies that data science is concerned with various techniques used in extracting structured or unstructured forms of data from many sources.

6.4 OUTPUT SCREENS

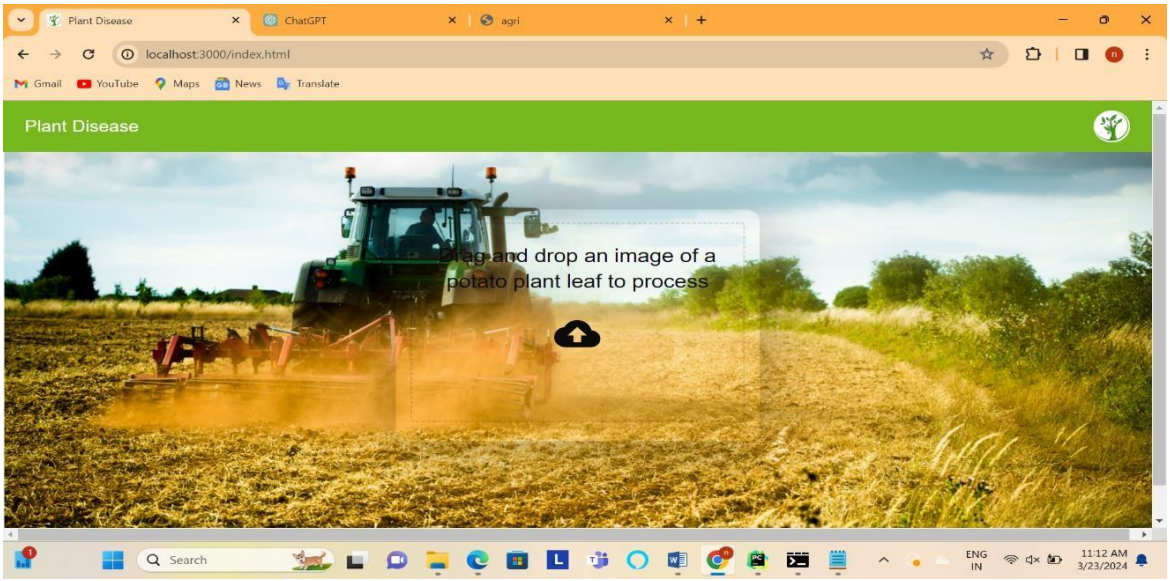
Home:

Screen.6.1: Home Page

Crop Selecting:

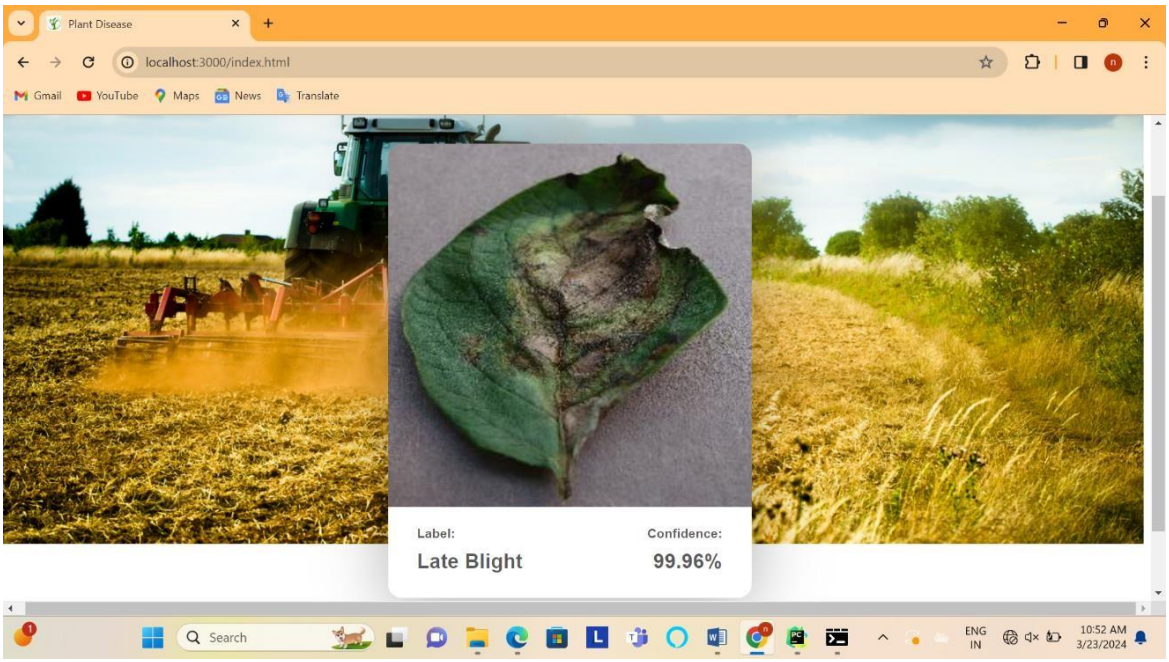
Screen.6.2: Crop selecting modules

Plant Disease:



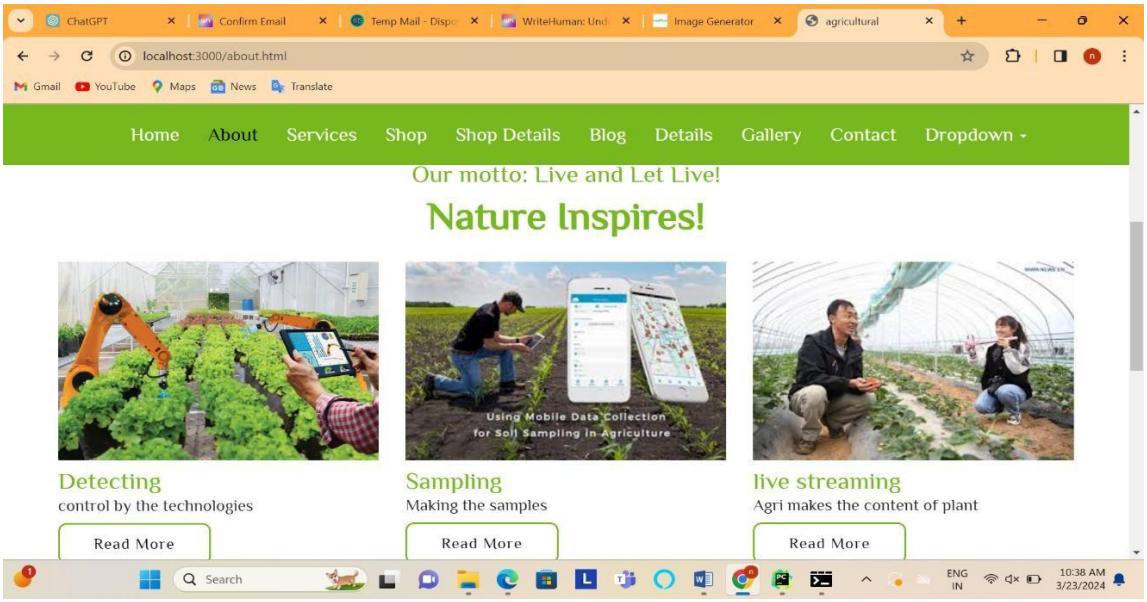
Screen.6.3: Plant disease classification page

Plant Disease Detection:



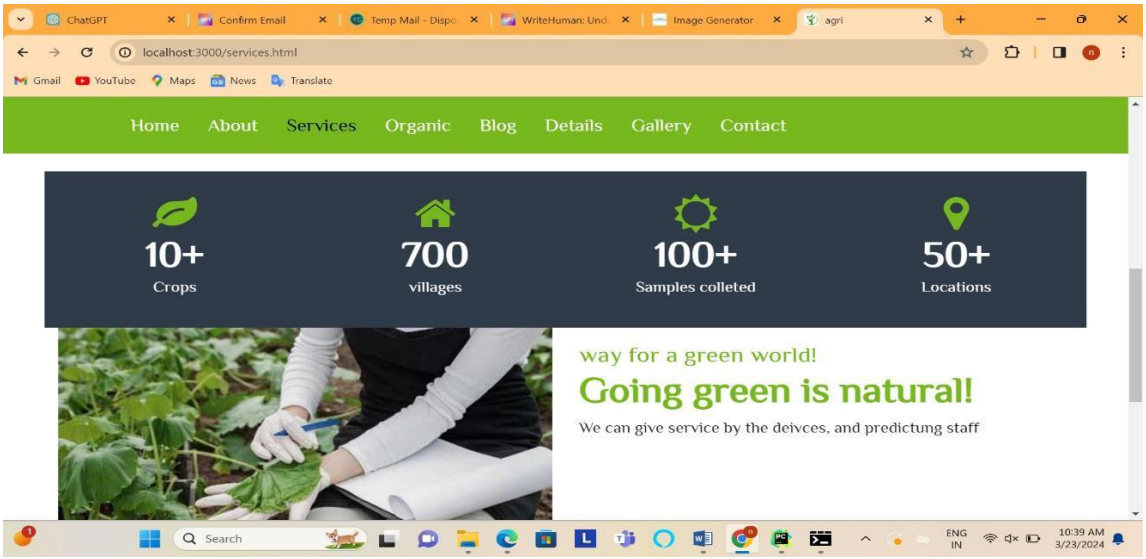
Screen.6.4: Plant disease detection with label & confidence

About:



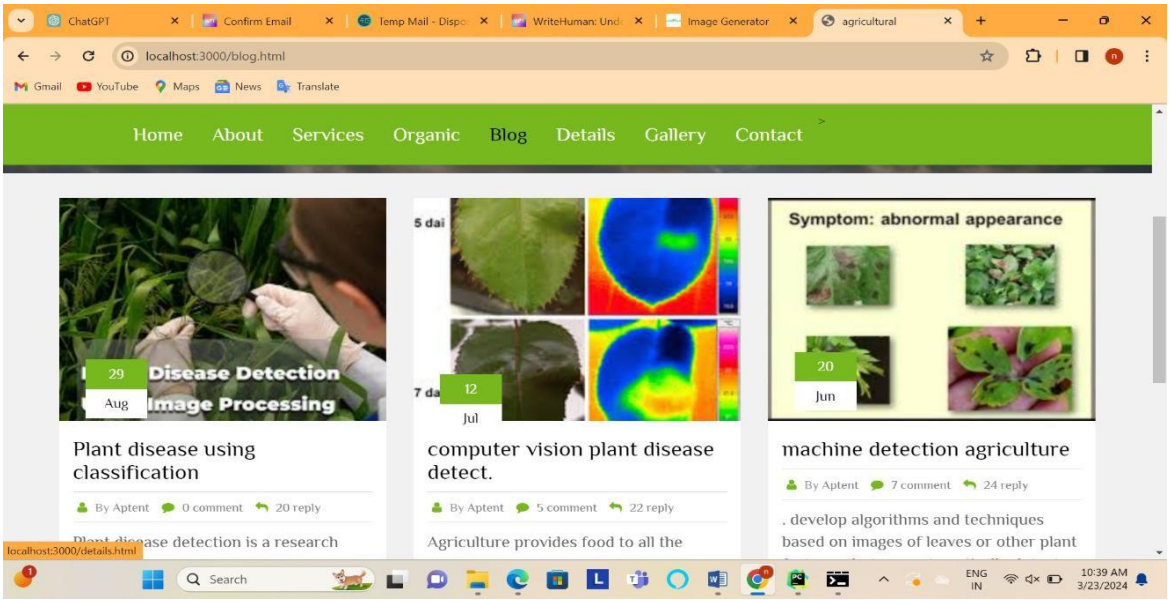
Screen.6.5: About page and tells about the detail process

Services:



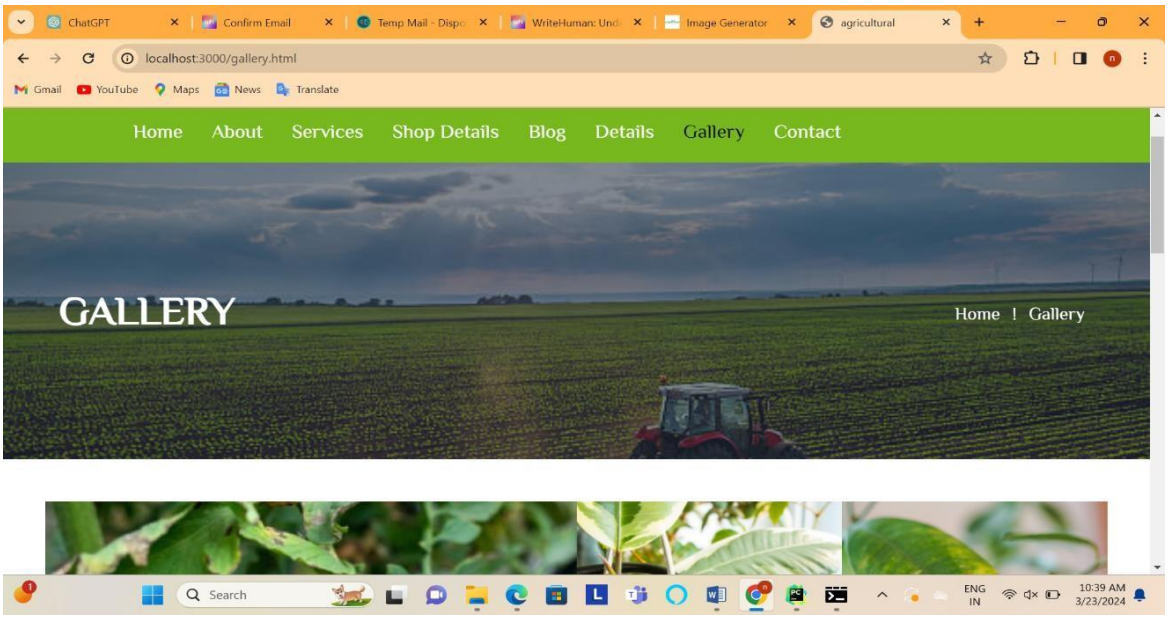
Screen.6.6: Service page have services done in sectors

Blog:



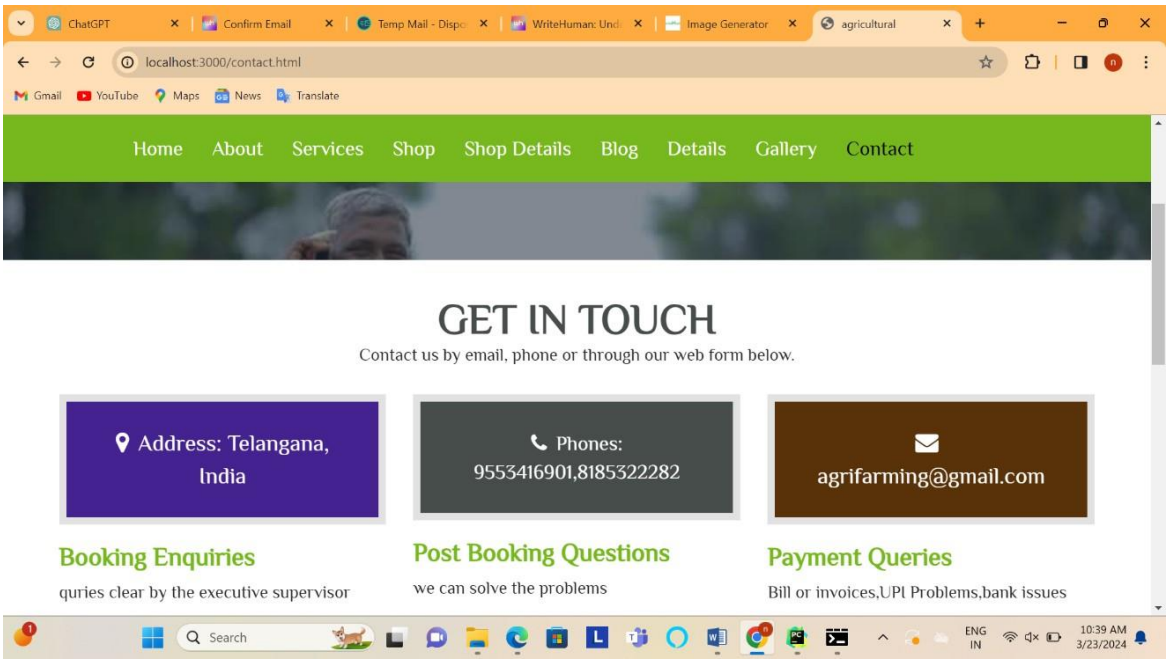
Screen.6.7: Blog page have different blog about their researches

Gallery:



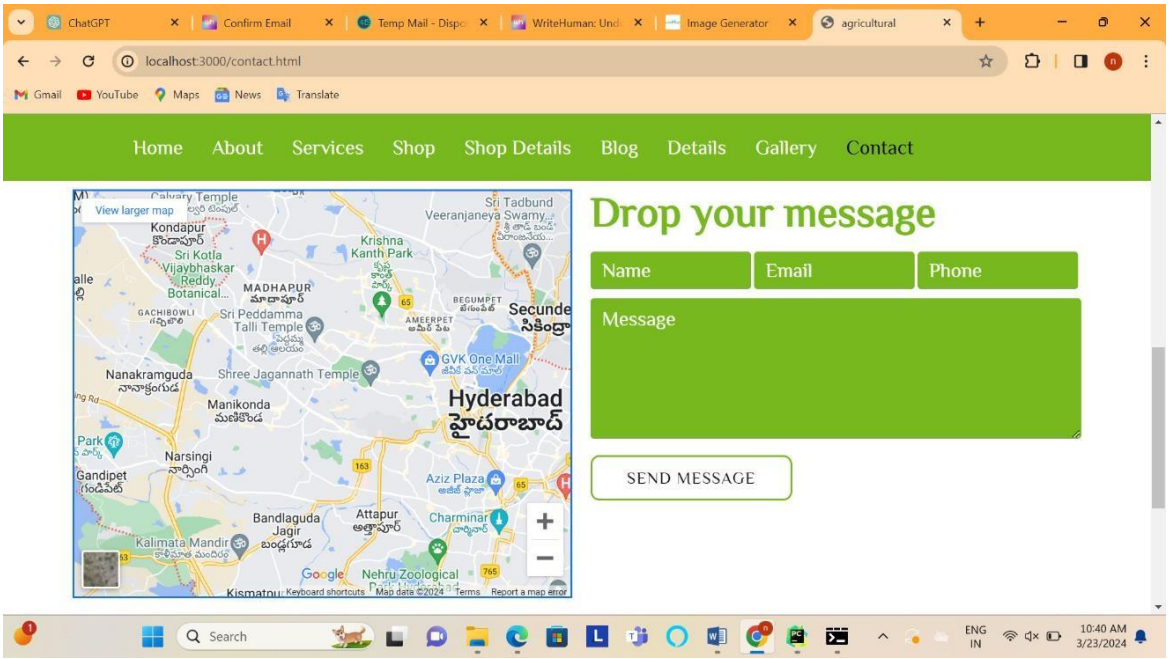
Screen.6.8: Gallery page maintain the pictures about achievement's & research

Contact:



Screen.6.9: Contact page have connection details

Location & Ping:



Screen.6.10: Location details and feedback space

7. TESTING & VALIDATION

7.1 Design of test cases and scenarios

7.1.1 Test Procedure

Black box Testing:

This type of software testing focuses on its functionality without going through the internal code. Testing is done like this in order to ensure that expected behaviors are maintained by the software. Black box testing is done by someone from outside who then gets involved in the development of the software's test cases in question. These may be either functional or non-functional tests, and valid/invalid input is analyzed to determine correct output.

Regression Testing:

Regression testing detects regression bugs. Regression testing is done not just for verifying program correctness but also frequently used to evaluate its output quality. This entails retesting the software after any changes have been made to ascertain whether all old features are functioning properly. It also helps in ensuring that no new errors were introduced through the new code as well as it resurrected once fixed issues.

Integration Testing:

In this kind of a test, individual software modules are tested when working together. By so doing, it tests for defects that exist between integrated units or modules interfaces and interactions. The process of developing software in which programs or modules are grouped together and tested as parts of many different combinations is known as integration testing.

GUI Testing:

GUI (Graphical User Interface) testing involves checking whether graphical elements of a software application operate appropriately and meet human visual aesthetics. Such may include elements such as buttons, menus, icons etc., which makes sure they are responsive and correctly aligned in place.

7.2 Test cases

Test Case Identifier	Description	Preconditions	Test Steps	Expected Result	Actual Result	Pass/Fail Status
TC1	Home Page Ads Display	Homepage is loaded successfully	1. Open the home page 2. Observe the display of ads	Ads related to tomato, pepper bell, and potato diseases are displayed	Same as Expected	Pass
TC2	Plant Type Selection	Homepage is loaded successfully	1. Open the home page 2. Click on a plant type option (e.g., tomato)	Selected plant type is highlighted/indicated	Same as Expected	Pass
TC3	Image Upload on Classification Page	User navigated to Classification Page	1. Upload or capture an image of a plant leaf 2. Proceed with the classification process	Uploaded/captured image is displayed	Same as Expected	Pass
TC4	Disease Prediction	Image is uploaded/captured on Classification Page	1. Click on 'predict' button 2. Wait for prediction results	Predicted disease label with confidence scores is displayed	Same as Expected	Pass
TC5	Classification Results Display	Prediction process is completed	1. View the classification results	Detected disease, certainty level, and recommended actions are displayed	Same as Expected	Pass
TC6	Clear Option Functionality	User is viewing classification results	1. Click on 'clear' or similar option	Classification results are cleared, and user is directed back to the previous page	Same as Expected	Pass
TC7	Navigation to About Page	User selects About Page from navigation	1. Click on About Page link	Information about project processes is displayed	Same as Expected	Pass
TC8	Navigation to Services Page	User selects Services Page from navigation	1. Click on Services Page link	Details about offered services are displayed	Same as Expected	Pass
TC9	Navigation to Organic Page	User selects Organic Page from navigation	1. Click on Organic Page link	Information about traditional organic farming practices is displayed	Same as Expected	Pass

Test Case Identifier	Description	Preconditions	Test Steps	Expected Result	Actual Result	Pass/Fail Status
TC10	Navigation to Blog Page	User selects Blog Page from navigation	1. Click on Blog Page link	Various blogs related to farming are displayed	Same as Expected	Pass
TC11	Navigation to Details Page	User selects Details Page from navigation	1. Click on Details Page link	Detailed explanations about crop diseases are displayed	Same as Expected	Pass
TC12	Navigation to Gallery Page	User selects Gallery Page from navigation	1. Click on Gallery Page link	Photos related to farming are displayed	Same as Expected	Pass
TC13	Navigation to Contact Page	User selects Contact Page from navigation	1. Click on Contact Page link	Contact information and Google map are displayed	Same as Expected	Pass
TC14	Feedback Form Submission	User fills out feedback form	1. Fill out the feedback form 2. Submit the form	Feedback is successfully submitted	Same as Expected	Pass

Table 7.1. Test cases of overall modules

7.2.1 Black Box Testing

Test Case Identifier	Description	Preconditions	Test Steps	Expected Result	Actual Result	Pass/Fail Status
BB1	Test Home Page Functionality	Application is launched successfully	1. Interact with the home page elements. 2. Click on ads and plant type options.	1. Ads should change on interaction. 2. Plant type options should navigate to respective classification pages.		
BB2	Test Classification Page Functionality	User navigates to the classification page	-	1. Upload or capture an image. 2. Click on predict button.	1. Predicted disease label with confidence scores is displayed.	Pass
BB3	Test About Page Navigation	User navigates to the about page	-	Verify information about classification, detection, sampling, and live		

Test Case Identifier	Description	Preconditions	Test Steps	Expected Result	Actual Result	Pass/Fail Status
				streaming processes.		
BB4	Test Services Page Navigation	User navigates to the services page	-	Verify details about current services and future developments.		
BB5	Test Organic Page Navigation	User navigates to the organic page	-	Verify information about traditional organic farming practices.		
BB6	Test Blog Page Navigation	User navigates to the blog page	-	Verify various blogs related to farming.		
BB7	Test Details Page Navigation	User navigates to the details page	-	Verify detailed explanations about crop diseases and detection model.		
BB8	Test Gallery Page Navigation	User navigates to the gallery page	-	Verify display of farming-related photos.		
BB9	Test Contact Page Navigation	User navigates to the contact page	-	Verify contact information and Google map display.		
BB10	Test Feedback Form Submission	User submits feedback through the form	Form is accessible	Fill out the form and submit	Feedback is successfully submitted	Pass

Table 7.2: Black box Testing

7.2.2 Regression Testing

Test Case Identifier	Description	Preconditions	Test Steps	Expected Result	Actual Result	Pass/Fail Status
RT1	Test Home Page Functionality	Application is launched successfully	Same as BB1	Same as BB1		Pass
RT2	Test Classification Page Functionality	User navigates to the classification page	Same as BB2	Same as BB2		Pass

Test Case Identifier	Description	Preconditions	Test Steps	Expected Result	Actual Result	Pass/Fail Status
RT3	Test About Page Navigation	User navigates to the about page	Same as BB3	Same as BB3		Pass
RT4	Test Services Page Navigation	User navigates to the services page	Same as BB4	Same as BB4		Pass
RT5	Test Organic Page Navigation	User navigates to the organic page	Same as BB5	Same as BB5		Pass
RT6	Test Blog Page Navigation	User navigates to the blog page	Same as BB6	Same as BB6		Pass
RT7	Test Details Page Navigation	User navigates to the details page	Same as BB7	Same as BB7		Pass
RT8	Test Gallery Page Navigation	User navigates to the gallery page	Same as BB8	Same as BB8		Pass
RT9	Test Contact Page Navigation	User navigates to the contact page	Same as BB9	Same as BB9		Pass
RT10	Test Feedback Form Submission	User submits feedback through the form	Same as BB10	Same as BB10		Pass

Table 7.3: Regression Testing

7.2.3 Integration Testing

Test Case Identifier	Description	Preconditions	Test Steps	Expected Result	Actual Result	Pass/Fail Status
IT1	Test Integration of Home Page with Classification Page	Home page is functional	Navigate to classification page from home page	Classification page should load without errors	Same as Expected	Pass
IT2	Test Integration of Feedback Form with Contact Page	Feedback form is accessible	Submit feedback through the form	Feedback should be successfully submitted	Same as Expected	Pass

Table 7.4: Integration Testing

7.2.4 GUI Testing

Test Case Identifier	Description	Preconditions	Test Steps	Expected Result	Actual Result	Pass/Fail Status
GUI1	Test User Interface Consistency	Application is launched successfully	Interact with different pages and elements	UI elements should be consistent across pages	Same as Expected	Pass
GUI2	Test Visual Appearance	Application is launched successfully	Navigate through different pages	Visuals should be appealing and coherent	Same as Expected	Pass

Table 7.5: GUI Testing

7.3 Conclusion

To conclude, this set of test cases and testing procedures provides a comprehensive framework for assuring the quality and functionality of the software application. The black box testing ascertains that home page navigation, sorting methods and other navigations are fully scrutinized hence ensuring that user interactions yield expected results. Regression testing also ensures that changes or updates to the software do not affect existing functionalities thereby maintaining the integrity of the application across releases. Integration tests further validate seamless interaction between various modules thereby confirming proper functioning of an integrated system. Lastly, GUI testing guarantees consistency and aesthetics in graphical user interface (GUI) throughout the application thus enhancing user experience. These rigorous testing approaches enable deployment with certainty by meeting users' expectations who demand dependable solutions to particular problems.

8. CONCLUSION

This work has successfully analyzed CNN models that are well suited for the accurate classification of 15 different types of plant diseases. Based on classification accuracy, sensitivity, specificity, and F1 score, standardization and evaluation of the recent convolutional neural networks were done using various techniques. From the performance analysis of the various pre-trained architectures. The proposed model achieved a classification accuracy of 99.81% and F1 score of 99.8%. A disease detection system is accurate and time efficient in plants with metrics comparable to existing state-of-the-art solution. This project employs modern techniques in deep learning field. The custom dataset was created by annotating it and then evaluating it consistently. As a result, all plant diseases can be detected with high precision. According to the necessity, crop prediction helps farmers grow crops. This can be used as a pre-processing phase for real-time applications which require disease detection in their pipeline. Further, there should also be strong, effective, efficient and flexible Interface system.

In future work will focus on problems associated with real-time data collection as well as creation of multi-object deep learning model capable of even detecting plant diseases from a bunch instead of an individual leaf. Moreover, we aim to roll out mobile application based on this study's trained model. It will facilitate real-time leaf disease identification for farmers and agriculture sector.

REFERENCES

[1] Naeem ullah, Javed ali khan, lubna abdulaziz alharbi,Asaf raza, wahab khan, and ijaz ahmad, “An Efficient Approach for Crops Pests Recognition and Classification Based on Novel DeepPestNet Deep Learning Model” 11 July 2002, doi: 10.1109/ACCESS.2022.3189676

[2] Lili li, Shujuan Zhang, and Bin wang “ Plant Disease Detection and Classification by Deep Learning” , April 8, 2021 , doi: 10.1109/ACCESS 2021.3069646

[3] YONG AI , Chong sun, Jun Tie , and Xiantao Cai ,”Research on Recognition Model of Crop Diseses and Insect pests based on Deep Learning in Harsh Environments”, 21,2020 doi: 10.1109/ACCESS.2020.3025325

[4] G. Shrestha, Deepsikha, M. Das and N. Dey, "Plant Disease Detection Using CNN," 2020 IEEE Applied Signal Processing Conference (ASPCON), 2020, pp. 109-113, doi:10.1109/ASPCON49795.2020.9276722.

[5] S. D. Khirade and A. B. Patil, “Plant Disease Identification with the help of Image Processing”, 2015 International Conference on Computing Communication Control and Automation, pp.768-771, doi: 10.1109/ICCUBEA.2015.153

[6] Sukanya C Madiwalar and Manohar V Wyawahare (2017). Comparative Analysis of Plant disease Identification Techniques .2017 International Conference on Data Management, Analytics and Innovation (ICDMAI),13-18, doi: 10.1109/ICDMAI.2017.8073478.

[7] Sarath D.M., Akhilesh S.A.Kumar R.M.G., Prakash C.(2020). Pomegranate Fungal Blight Detection Based on Color Images using Machine Learning. Sensors (Basel) ;20(11):3136.

[8] P. Venkatasaichandrakanth and M. Iyapparaja, “ Pest Detection and classification in Peanut Crops Using CNN, MFO and Evita Algorithm” 30 May, 2023 , doi: 10.1109/ACCESS.2023.3281508

[9] Rahul Kundu, Usha Chauhan, S.P.S Chauhan, “Plant Leaf Disease Detection using Image Processing” 2022, doi: 10.1109/ICIPTM54933.2022.9754170

[10] Melike Sardogan, Adem Tuncer, Yunus Ozen, “Plant Leaf Disease Detection and Classification Based on CNN with LVQ Algorithm” 2018, doi: 10.1109/UBMK.2018.8566635

[11] Sandeep Kumar, KMVV Prasad, A. Srilekha, T. Suman, B. Pranav Rao, J. Naga Vamshi Krishna, “Leaf Disease Detection and Classification based on Machine Learning”, 09-10 October 2020 doi: 10.1109/ICSTCEE49637.2020.9277379

[12] Santhosh S. Kumar; B.K. Raghavendra, “Diseases Detection of Various Plant Leaf Using Image Processing Techniques”, 06 June 2019, doi: 10.1109/ICACCS.2019.8728325